

Vulnerabilidades de Software

**Naturaleza, clasificación,
minimización y búsqueda**

PRESENTADO POR:
Mikel Iturbe Urretxa

ENTIDAD
Mondragon Unibertsitatea

MÓDULO
Avanzado

Presentación

- Mikel Iturbe Urretxa
- iturbe.info
- Docente en el [Máster Universitario en Análisis de datos, Ciberseguridad y Computación en la Nube](#) y en el [Máster de formación permanente Online en Ciberseguridad](#)
 - + Seguridad en Grado en Ingeniería informática, sesiones de formación a empresas...
- Años de experiencia en investigación y transferencia (proyectos de ciberseguridad con empresas)
- Particularmente en relación con este módulo:
 - Búsqueda de vulnerabilidades (auditorías, fuzzing...)
 - Diseño e implantación de sistemas de gestión de vulnerabilidades dentro de ciclos de vida de producto

Mondragon Unibertsitatea



**Mondragon
Unibertsitatea**

**Escuela Politécnica
Superior**

- Universidad privada
- Escuela Politécnica Superior:
 - Cooperativa de iniciativa social sin ánimo de lucro
 - Fundación 1943 como escuela técnica
 - Desde 1997 Escuela Politécnica Superior de Mondragon Unibertsitatea
- Grupo de investigación de Análisis de Datos y Ciberseguridad
 - Pioneros en investigación en ciberseguridad industrial (antes que Stuxnet!)
 - 60% actividad investigadora y 40% actividad de transferencia

¿Qué es una vulnerabilidad de software?

¿Qué es una vulnerabilidad?

[En un post de INCIBE](#), podemos leer la siguiente definición:

Una vulnerabilidad (...) es una debilidad o fallo en un sistema de información que pone en riesgo la seguridad de la información pudiendo permitir que un atacante pueda comprometer la integridad, disponibilidad o confidencialidad de la misma, por lo que es necesario encontrarlas y eliminarlas lo antes posible. Estos «agujeros» pueden tener distintos orígenes por ejemplo: fallos de diseño, errores de configuración o carencias de procedimientos.

Objetivo para hoy

- Profundizar en la definición anterior y ayudaros a cumplir con la necesidad expuesta
- Tratar de entender mejor que son las vulnerabilidades
- Como las podemos minimizar,
 - A la hora de parchear software de otros
 - Y minimizar su existencia en nuestros sistemas

Agenda

- ~~Qué es una vulnerabilidad.~~
- Tipos de vulnerabilidades
- Identificación de vulnerabilidades
- Clasificación y priorización de vulnerabilidades
- Cómo evitar vulnerabilidades
 - Requisitos, modelado de amenazas y validación de entradas
- Búsqueda de vulnerabilidades
 - Sesión teórico-práctica

Tipos de vulnerabilidades

Ejemplos de vulnerabilidades

- No quiero entrar en la explotación a bajo nivel de diferentes vulnerabilidades
- Ese aspecto suele estar bastante presente en la formación en ciberseguridad, al menos a nivel teórico.
- Con la definición que hemos visto antes, y con lo que ya sabéis, es suficiente.

Tipos de vulnerabilidades

Year	Overflow	Memory Corruption	Sql Injection	XSS	Directory Traversal	File Inclusion	CSRF	XXE	SSRF	Open Redirect	Input Validation
2016	418	1096	85	476	90	4	85	39	15	28	0
2017	945	1340	485	1427	252	13	326	101	55	69	1
2018	2055	1721	503	2039	569	111	479	187	118	85	1207
2019	1195	1975	544	2387	485	125	559	136	103	121	891
2020	1214	1829	463	2197	435	107	414	119	130	100	798
2021	1649	2463	740	2717	545	89	520	126	187	133	654
2022	1753	2800	1760	3362	680	85	766	123	228	137	649
2023	1589	1972	2111	5091	736	105	1389	124	237	167	509
2024	1722	2353	2647	7431	911	244	1428	110	372	113	86
2025	2249	2894	3945	8730	1053	672	1950	115	558	166	0
2026	2130	2206	2193	3898	1083	550	671	59	761	134	0
Total	16919	22649	15476	39755	6839	2105	8587	1239	2764	1253	4795

Impacto de vulnerabilidades

Vulnerabilities by impact types

Year	Code Execution	Bypass	Privilege Escalation	Denial of Service	Information Leak
2016	1239	1	149	2050	102
2017	1870	340	538	3372	470
2018	1728	641	825	2207	1390
2019	1556	654	898	1697	1313
2020	1691	793	1364	1677	1086
2021	2087	766	1079	2297	909
2022	2067	809	1392	2437	1103
2023	2580	811	1278	2560	1379
2024	3962	628	1052	2470	894
2025	3039	637	1019	2526	742
2026	2593	579	827	2878	595
Total	24412	6659	10421	26171	9983

Fuente: cvedetails.com

Identificación de vulnerabilidades

Identificación de vulnerabilidades

- De cara al conocimiento, hay tres tipos de vulnerabilidades
 - Vulnerabilidades no descubiertas
 - Vulnerabilidades descubiertas cuya existencia es pública y conocida
 - Vulnerabilidades descubiertas cuya existencia es limitada y es desconocida para el público en general
- ¿Públicas? ¡Quiz!

Vulnerabilidades

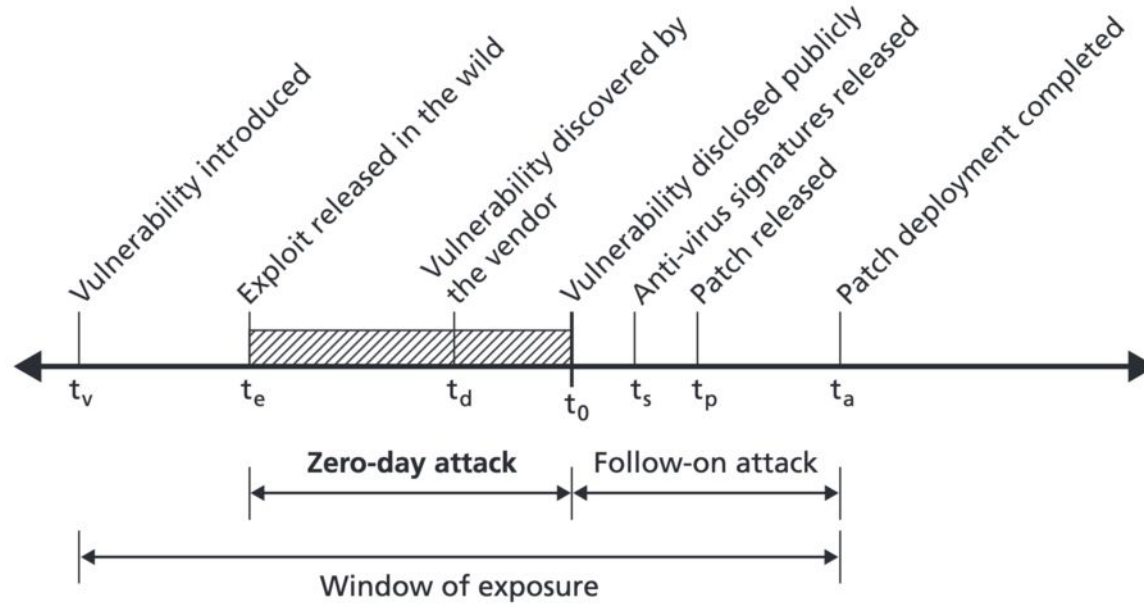
wooclapp.com

Código:

UXPJDA



Ciclo de vida de vulnerabilidades



Identificación de vulnerabilidades

- Todas las vulnerabilidades públicas están indexadas en algún repositorio de vulnerabilidades, públicas o pertenecientes a empresas.
 - China National Vulnerability Database (CNVD)
 - Common Vulnerabilities and Exposures (CVE)
 - European Union Vulnerability Database (EUVD)

Identificación de vulnerabilidades

- Totolink router buffer overflow
- CVE-2024-7177
- CNVD-2025-13900
- EUVD-2024-48145

CVE-2024-7177 PUBLISHED

Required CVE Record Information

CNA: VulDB

Published: 2024-07-29 Updated: 2024-07-29

Title: TOTOLINK A3600R Cstecgi.Cgi SetLanguageCfg Buffer Overflow

TOTOLINK A3600R缓冲区溢出漏洞

★ 关注(0)

CNVD-ID	CNVD-2025-13900
公开日期	2025-06-27
危害级别	高 (AV:N/AC:L/Au:S/C:C/I:C/A:C)
影响产品	TOTOLINK A3600R 4.1.2cu.5182_B20201102
CVE ID	CVE-2024-7177
漏洞描述	TOTOLINK A3600R是一款由中国吉翁电子（TOTOLINK）公司生产的6天线1200M无线路由器。 TOTOLINK A3600R存在缓冲区溢出漏洞。该漏洞源于/cgi-bin/cstecgi.cgi文件的setLanguageCfg函数中的langType参数。攻击者可利用该漏洞导致任意代码执行、设备控制权获取，甚至使设备崩溃或无法正常工作等严重后果。
漏洞类型	通用型漏洞
参考链接	https://github.com/abcdefg-png/ToT-vulnerable/blob/main/TOTOLINK/A3600R/setLanguageCfg.md https://vuldb.com/?submit.378044 https://vuldb.com/?ctiid.272598
漏洞解决方案	目前厂商尚未发布升级程序修复该安全问题，详情见厂商官网： https://www.totolink.net/
厂商补丁	(无补丁信息)
验证信息	(暂无验证信息)

Home / Vulnerability List / EUVD-2024-48145

EUVD-2024-48145 Back to the vulnerability search

Severity

CVSS Base Score: 8.7 (x4.0)

(CVSS:3.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:N/SN:N) 1.12 %

Alternative IDs

CVE-2024-7177

Summary

A vulnerability was found in TOTOLINK A3600R 4.1.2cu.5182_B20201102. It has been classified as critical. Affected is the function setLanguageCfg of the file cgi-bin/cstecgi.cgi. The manipulation of the argument langType leads to buffer overflow. It is possible to launch the attack remotely. The exploit has been disclosed to the public and may be used. VDB-272598 is the identifier assigned to this vulnerability. NOTE: The vendor was contacted early about this disclosure but did not respond in any way.

Affected Product

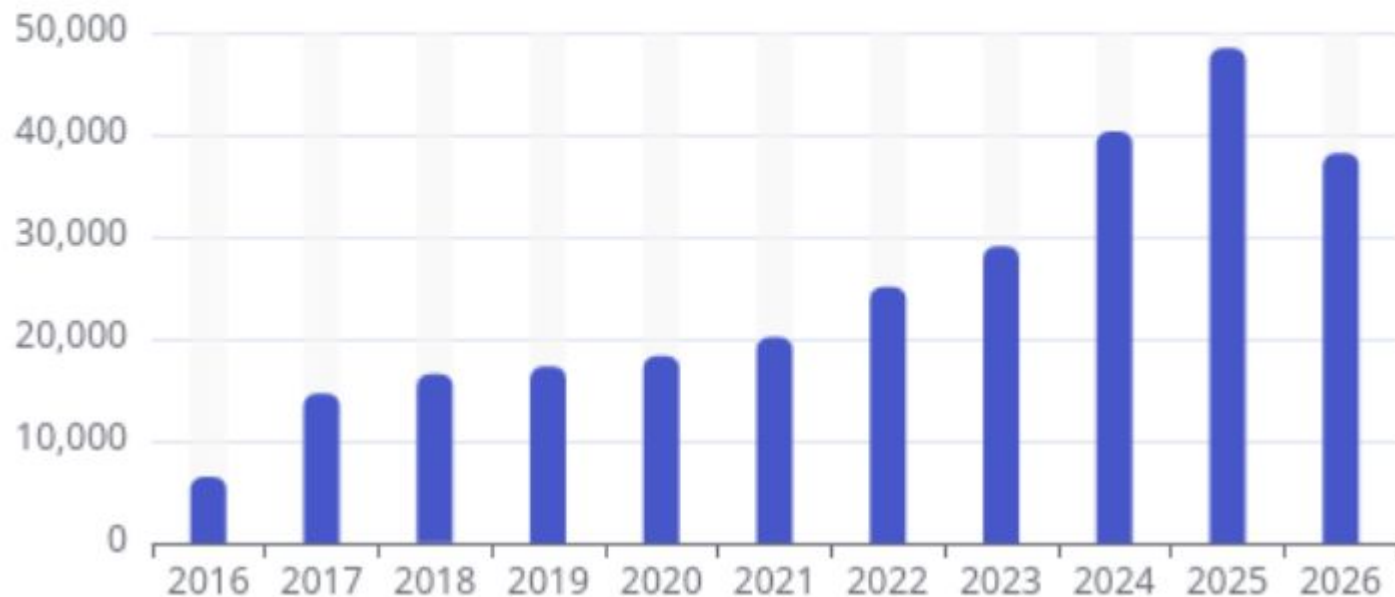
Vendor:	Product:	Version:
Totolink	A3600R	4.1.2cu.5182_B20201102

Identificación de vulnerabilidades: CVE

- CVE (Common Vulnerabilities and Exposures) es el sistema de referencia más utilizado a nivel mundial para identificar vulnerabilidades.
- Fue creado por MITRE en 1999 y está respaldado por el gobierno de EE.UU. a través de la CISA.
- Un CVE contiene:
 - Un número identificador único con el formato CVE-AÑO-NÚMERO (por ejemplo: CVE-2023-12345).
 - Una breve descripción del problema
 - Un enlace a fuentes que detallan la vulnerabilidad
- El objetivo de CVE es ofrecer un lenguaje común entre herramientas y equipos de seguridad, evitando confusión y mejorando la coordinación en la respuesta a incidentes.

Identificación de vulnerabilidades: CVE

Number of CVEs by year



Otros identificadores útiles

- Otros identificadores útiles.
 - Common Platform Enumeration (CPE): Identifica la plataforma vulnerable de forma inequívoca
 - Software y versión puede ser ambiguo (sobre qué plataforma etc.)
 - Common Weakness Enumeration (CWE): Identifica el tipo de debilidad que originó la vulnerabilidad.
 - Common Attack Pattern Enumeration and Classification (CAPEC): Identifica el/los patrones de ataque utilizado en su explotación

Otros identificadores útiles (Ejemplo)

- Vulnerabilidad CVE: [CVE-2024-31210](#)

Vulnerability Details : [CVE-2024-31210](#)

WordPress is an open publishing platform for the Web. It's possible for a file of a type other than a zip file to be submitted as a new plugin by an administrative user on the Plugins → Add New → Upload Plugin screen in WordPress. If FTP credentials are requested for installation (in order to move the file into place outside of the `uploads` directory) then the uploaded file remains temporary available in the Media Library despite it not being allowed. If the `DISALLOW\_FILE\_EDIT` constant is set to `true` on the site and FTP credentials are required when uploading a new theme or plugin, then this technically allows an RCE when the user would otherwise have no means of executing arbitrary PHP code. This issue only affects Administrator level users on single site installations, and Super Admin level users on Multisite installations where it's otherwise expected that the user does not have permission to upload or execute arbitrary PHP code. Lower level users are not affected. Sites where the `DISALLOW\_FILE\_MODS` constant is set to `true` are not affected. Sites where an administrative user either does not need to enter FTP credentials or they have access to the valid FTP credentials, are not affected. The issue was fixed in WordPress 6.4.3 on January 30, 2024 and backported to versions 6.3.3, 6.2.4, 6.1.5, 6.0.7, 5.9.9, 5.8.9, 5.7.11, 5.6.13, 5.5.14, 5.4.15, 5.3.17, 5.2.20, 5.1.18, 5.0.21, 4.9.25, 2.8.24, 4.7.28, 4.6.28, 4.5.31, 4.4.32, 4.3.33, 4.2.37, and 4.1.40. A workaround is available. If the `DISALLOW\_FILE\_MODS` constant is defined as `true` then it will not be possible for any user to upload a plugin and therefore this issue will not be exploitable.

Published 2024-04-04 23:15:16 Updated 2024-04-05 12:40:53 Source [GitHub, Inc.](#)

View at [NVD](#), [CVE.org](#)

Otros identificadores útiles (Ejemplo)

- Plataforma afectada (CPE): [cpe:2.3:a:wordpress:wordpress:3.8:-:*:*:*:*:*](#)

CPE Summary

[Return to Search Listing](#)

CPE Names

Version 2.3: **cpe:2.3:a:wordpress:wordpress:3.8:-:*:*:*:*:***

Version 2.2: **cpe:/a:wordpress:wordpress:3.8:-**

[Read information about CPE Name encoding](#)

This CPE has deprecated the following CPE(s):

[cpe:2.3:a:wordpress:wordpress:3.8:-:*:*:*:*:*](#)



CPE NAME COMPONENTS [SELECT A COMPONENT TO SEARCH FOR SIMILAR CPES](#)

Part: a
Vendor: wordpress
Product: wordpress
Version: 3.8
Update: -

QUICK INFO

Created On: 06/10/2019
Last Modified On: 06/10/2019

Metadata

Titles:

Text

Locale

WordPress 3.8

en_US

Otros identificadores útiles (Ejemplo)

- Debilidad que hace posible la vulnerabilidad (CWE): [CWE-434 : Unrestricted Upload of File with Dangerous Type](#)

CWE-434 : Unrestricted Upload of File with Dangerous Type

The product allows the upload or transfer of dangerous file types that are automatically processed within its environment.

Created: 2006-07-19 Updated: 2024-11-19 Source: <https://cwe.mitre.org/data/definitions/434.html> 

[Vulnerabilities related to CWE-434](#)

Otros identificadores útiles (Ejemplo)

- Patrón de ataque utilizado en su explotación: [CAPEC-1 : Accessing Functionality Not Properly Constrained by ACLs](#)

CAPEC-1 : Accessing Functionality Not Properly Constrained by ACLs

In applications, particularly web applications, access to functionality is mitigated by an authorization framework. This framework maps Access Control Lists (ACLs) to elements of the application's functionality; particularly URL's for web apps. In the case that the administrator failed to specify an ACL for a particular element, an attacker may be able to access it with impunity. An attacker with the ability to access functionality not properly constrained by ACLs can obtain sensitive information and possibly compromise the entire application. Such an attacker can access resources that must be available only to users at a higher privilege level, can access management sections of the application, or can run queries for data that they otherwise not supposed to.

<https://capec.mitre.org/data/definitions/1.html> 

Clasificación y priorización de vulnerabilidades

Vulnerabilidades

- Sabemos que las vulnerabilidades no son buenas
- Deberíamos parchearlas
- Tenemos recursos limitados (tiempo, dinero, downtime, criticidad de activos...)
- ¿Qué vulnerabilidades deberíamos parchear? ¡Quiz!

Vulnerabilidades

wooclap.com

Código:

UXPJDA



Vulnerabilidades

- Sabemos que las vulnerabilidades no son buenas
- Deberíamos parchearlas
- Tenemos recursos limitados (tiempo, dinero, downtime, criticidad de activos...)
- ¿Qué vulnerabilidades deberíamos parchear?
 - ¿Las más graves?
 - ¿Las más explotables?

Vulnerabilidades

- No todas las vulnerabilidades son iguales, hay algunas que son más graves.
- Esa gravedad es clave para poder priorizar el esfuerzo en su parcheo
- Clasificaciones
 - Common Vulnerability Scoring System (CVSS)
 - Exploit Prediction Scoring System (EPSS)
 - Known Exploited Vulnerabilities (KEV)

Common Vulnerability Scoring System (CVSS)

- Es un framework estándar para determinar la **severidad** de la vulnerabilidades de software
- Última versión CVSS v4.0 (2023). Más precisión que v3.1 (aunque éste se siga utilizando)
- Puntuación:
 - 0.0 Ninguna a 10 (crítica)
- Métrica a partir de tres aspectos:
 - Puntuación Base: Severidad intrínseca (vector de ataque, impact...)
 - Puntuación temporal: Factores que cambian con el tiempo ([p.ej.](#), disponibilidad de exploit).
 - Puntuación de entorno: Impacto en el entorno de la vulnerabilidad.
- Es calculable para todas las vulnerabilidades
 - Calculadora: <https://www.first.org/cvss/calculator/3-1>

Exploit Prediction Scoring System (EPSS)

- Puntuación de predicción de la probabilidad de que una vulnerabilidad sea explotada en el corto plazo.
- Versión 4 (2025)
- Calculado por un modelo de ML
 - Basado con información de inteligencia de amenazas, datos históricos
- Puntuación
 - 0,0 (mínima probabilidad) - 1.0 (máxima probabilidad). A veces con %
- Sólo disponibles para vulnerabilidades públicas

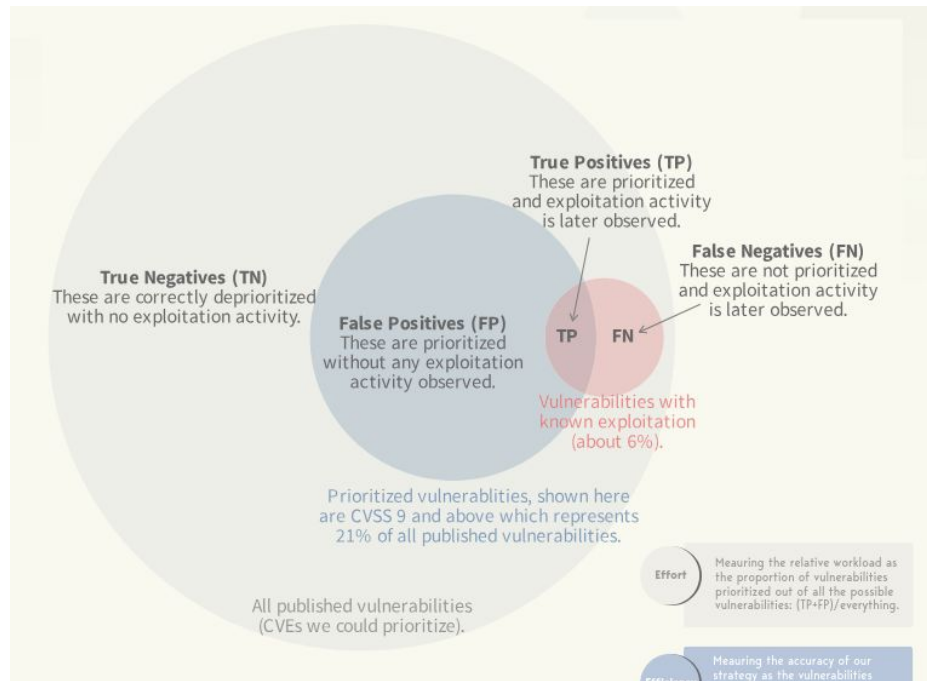
Known Exploited Vulnerabilities (KEV)

- Lista de vulnerabilidades confirmadas que están siendo explotadas
- Publicada por CISA (Cybersecurity and Infrastructure Security Agency)
- Puede haber vulnerabilidades nuevas o antiguas, ya que se actualiza con las pruebas de explotación, no por fecha de publicación.
- Catálogo: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

Qué vulnerabilidades deberíamos de parchear

- Métricas:
 - Coverage: ¿Qué porcentaje de todas las vulnerabilidades explotadas han sido correctamente priorizadas?
 - Eficiencia/Recall: ¿Qué porcentaje de las vulnerabilidades priorizadas han sido realmente explotadas?
 - Esfuerzo: Trabajo que supone implementar la estrategia de priorización (tradeoff entre las anteriores). ¿Qué porcentaje de todas las vulnerabilidades hemos elegido priorizar?

Qué vulnerabilidades deberíamos de parchear



Fuente: [Cyentia Institute: A Visual Exploration of Exploits in the Wild \(2024\)](#)

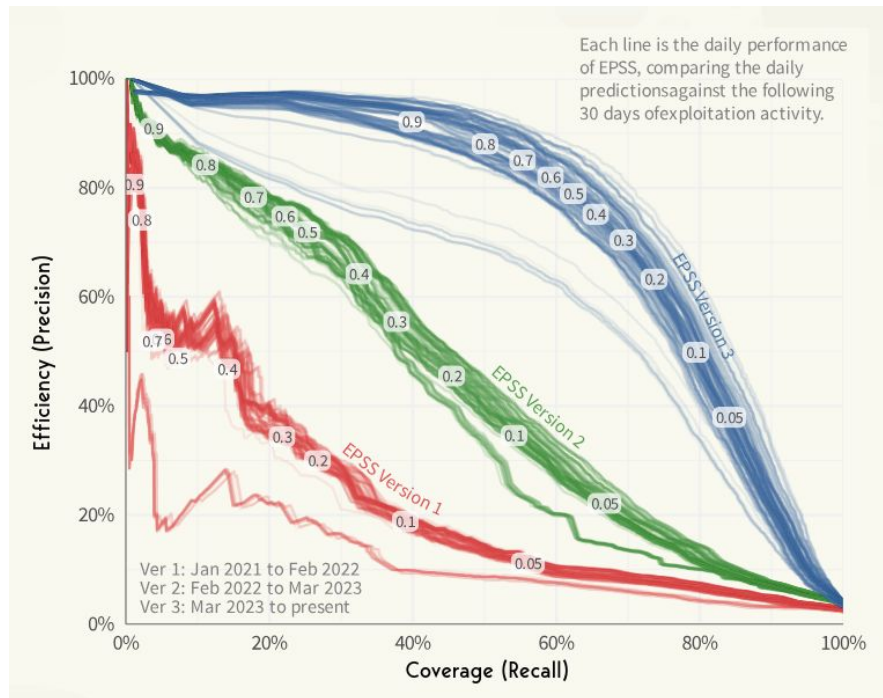
Qué vulnerabilidades deberíamos de parchear

- CVSS?

- CVSS mide la severidad y/o gravedad, no la explotabilidad. Pueden estar relacionados, o no.
- Solo el ~37% de las vulnerabilidades con score CVSS mayor que 9 tienen exploits conocidos (2024).
- Es habitual recomendar (y a veces, obligar) el parcheo de vulnerabilidades con un score mayor que 7.
 - Mitiga la mayoría (63%) de vulnerabilidades explotadas (las más graves, además)
 - Pero la eficiencia es de 10%. Es decir, el 90% de las vulnerabilidades parcheadas no tienen son explotadas. Se priorizan muchas vulnerabilidades sin exploits.

Qué vulnerabilidades deberíamos de parchear

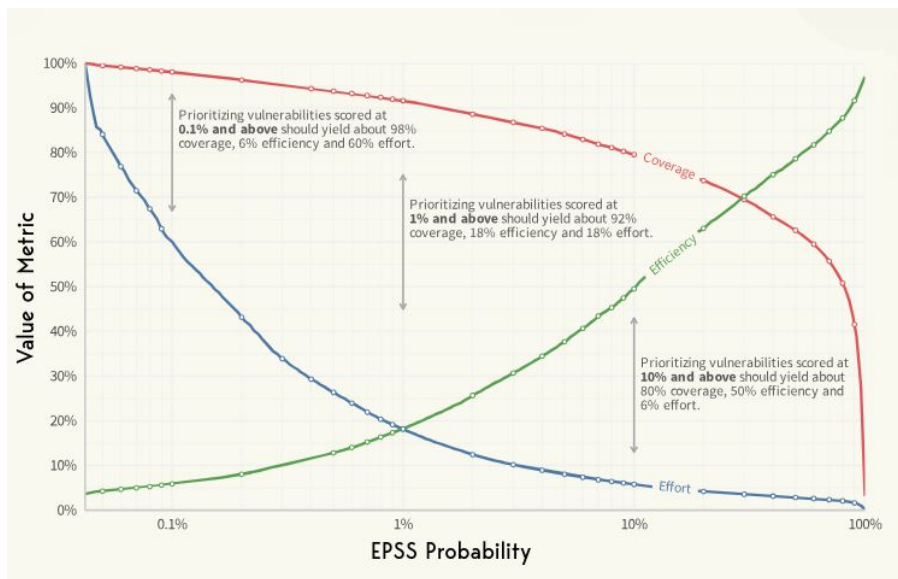
- EPSS?
 - Su versión

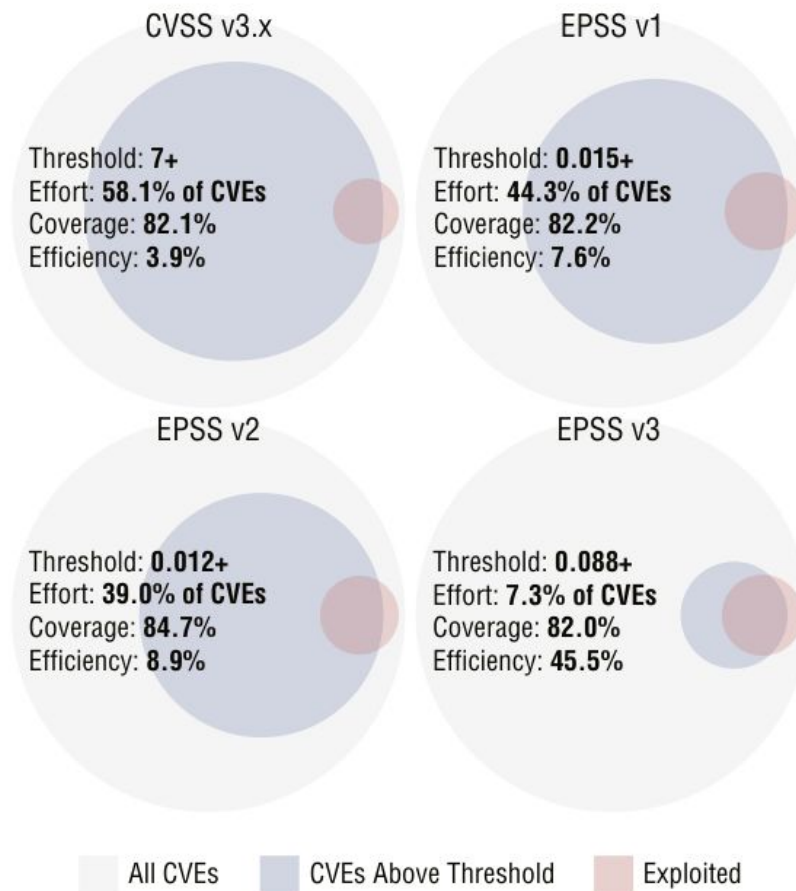


Qué vulnerabilidades deberíamos de parchear

- EPSS?

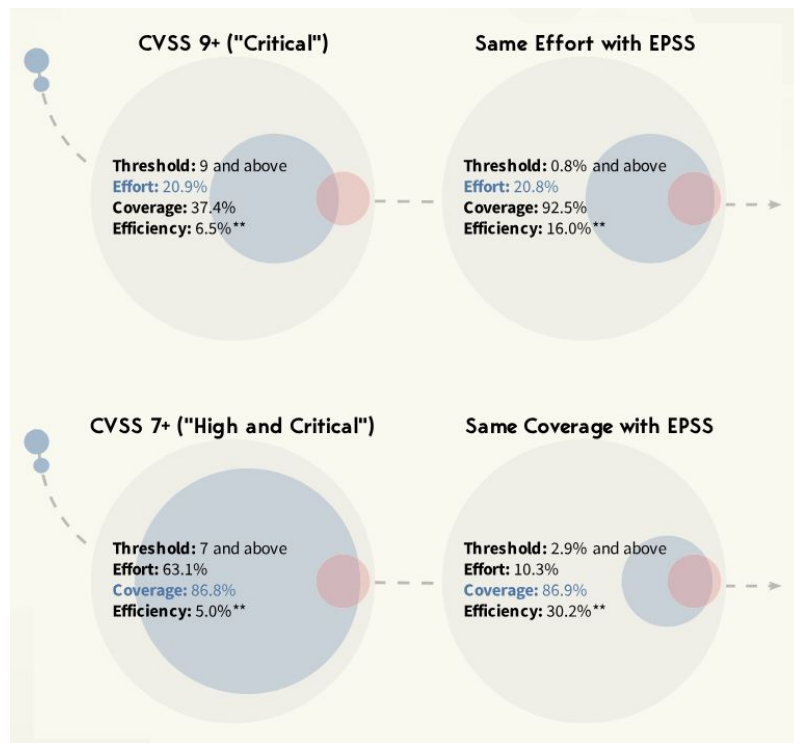
- ¿A partir de qué probabilidad empezamos a parchear?





Fuente: Hughes, C., & Robinson, N. (2024). Effective Vulnerability Management: Managing Risk in the Vulnerable Digital Ecosystem. John Wiley & Sons.

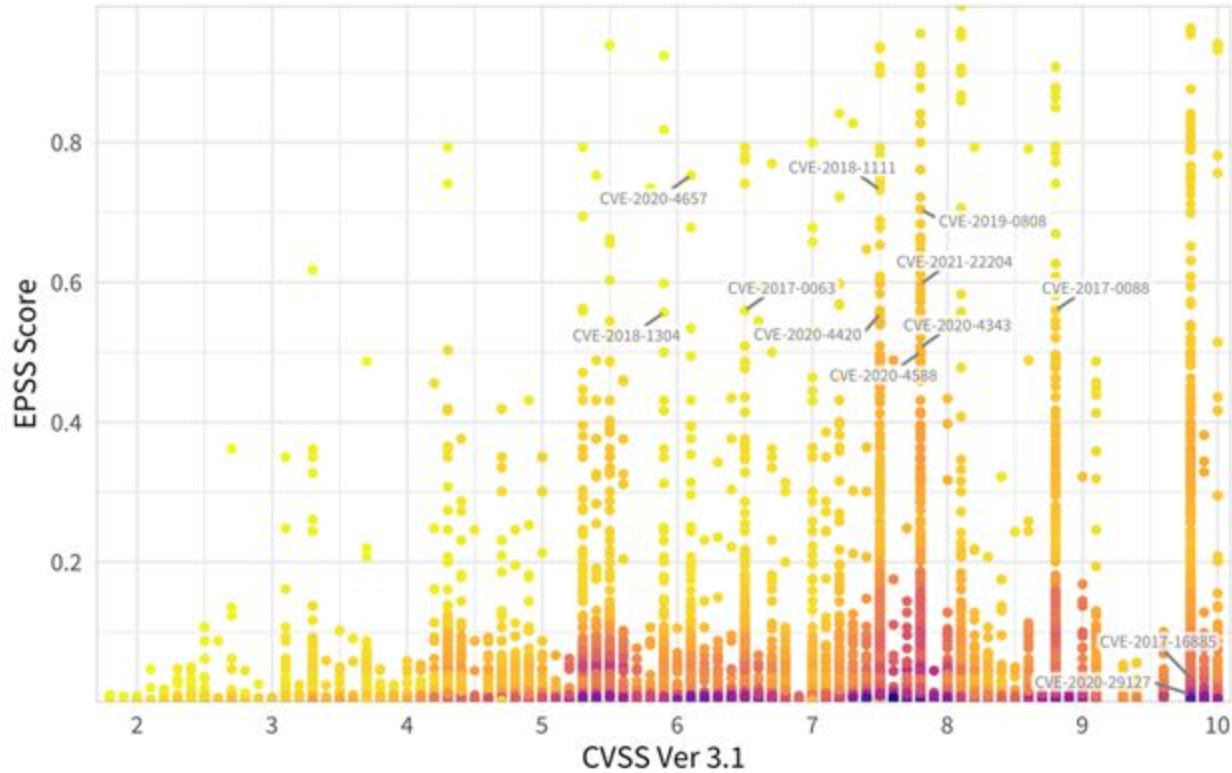
Qué vulnerabilidades deberíamos de parchear



Fuente: [Cyentia Institute: A Visual Exploration of Exploits in the Wild \(2024\)](#)

EPSS score compared to CVSS Base Score (NVD)

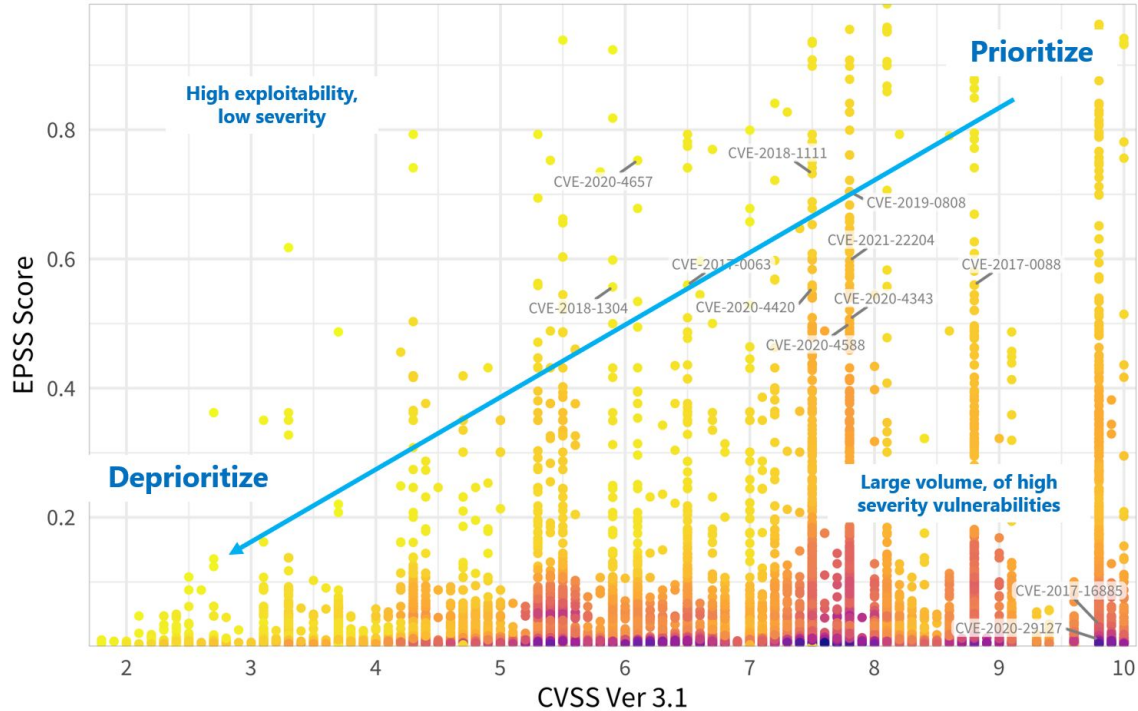
Point density is represented by color, yellow is less dense going through red to a deep purple for the most dense areas. Labeling a random sample of CVEs with higher values for reference.



Source: https://first.org/epss/data_stats, 2021-05-16

EPSS score compared to CVSS Base Score (NVD)

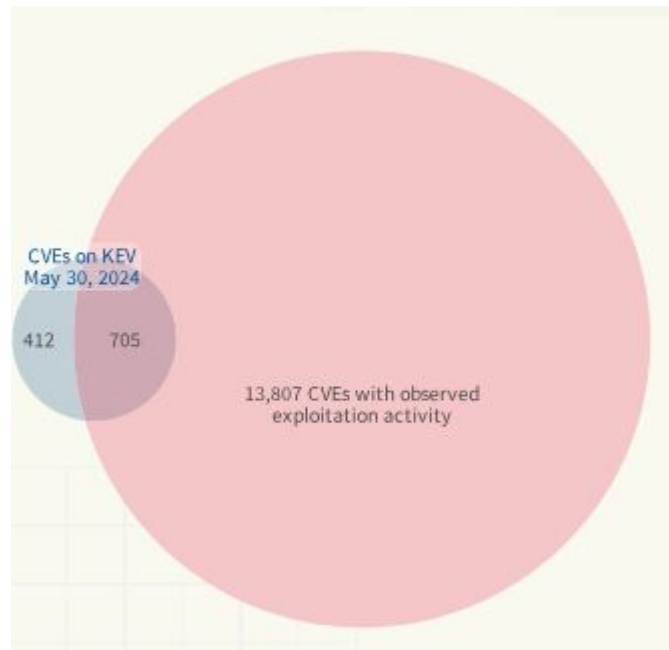
Point density is represented by color, yellow is less dense going through red to a deep purple for the most dense areas. Labeling a random sample of CVEs with higher values for reference.



Source: https://first.org/epss/data_stats, 2021-05-16

Qué vulnerabilidades deberíamos de parchear

- KEV?
 - Buen inicio como mínimo, maximizando el impacto del esfuerzo



Cómo evitar vulnerabilidades

El origen y coste de las vulnerabilidades

Requisitos y diseño	Implementación	Integración/Testing	Testeo en cliente	Post-producción
1x	5x	10x	15x	30x

Fuente: *Minimizing code defects to improve software quality and lower development costs*. IBM Rational Software Analyzer and IBM Rational PurifyPlus software. October 2008.

El origen y el coste de las vulnerabilidades

- ¿Cuándo se introducen las vulnerabilidades?

Vulnerabilidades

wooclapp.com

Código:

UXPJDA



El origen y el coste de las vulnerabilidades

- La mayoría de las vulnerabilidades se introducen en las fases iniciales de desarrollo, en requisitos y diseño.
- ¿Por qué creéis que es así?
- Por ello, en esta sesión nos vamos a centrar en esas fases iniciales
 - Para las de después, acordáos de la sesión de SecDevOps.

Vulnerabilidades y pasta



Cómo evitar vulnerabilidades

Requisitos

Requisitos de seguridad

- ¿Por dónde empezar? Dos pasos principales.
 - Identificación de activos de software o datos (módulos, DBs etc.)
 - Asignar niveles de seguridad
- Muchas veces, los requisitos están definidos en estándares de seguridad y/o en whitepapers
 - P.ej. OWASP's ASVS, PCI DSS, ISO/IEC 27034-3:2018, IEC 62443-4-2...
- Los requisitos deben ser:
 - Testeables y medibles
 - Claros, no ambiguos y consistentes

Requisitos de seguridad: ASVS

- Conjunto de requisitos de seguridad para aplicaciones web
 - También se pueden utilizar en otros escenarios
- Versión inicial completa: 1.0 (2009)
- Versión más reciente: 5.0.0 (Mayo 2025)
- ~350 requisitos (287 en la versión anterior)
- Disponible en PDF (123 pages), CSV, JSON, SQL...

Requisitos de seguridad: ASVS

- Niveles de seguridad
- Nivel 1 (Mínimo):
 - Nivel de seguridad bajo. Se puede comprobar de forma externa y se podría comprometer
 - Amenazas: Atacantes utilizando técnicas básicas y de bajo nivel técnico.
 - ~25% del total
- Nivel 2 (Estándar):
 - Aplicaciones que tienen datos sensibles. Es el nivel recomendado para la mayoría de aplicaciones.
 - Amenazas: Atacantes con recursos y motivados, centrados en ese objetivo y utilizando herramientas especializadas
 - ~%50 del total
- Nivel 3 (Crítico):
 - Las aplicaciones más críticas: infraestructura críticas o aplicaciones militares, médicas, financieras...
 - Tiene más impacton el diseño
 - Amenazas: Atacantes con gran nivel de recursos
 - ~30% del total

Requisitos de seguridad

- Hay empresas que también pueden definir las suyas

 Outsourcing

Web Application Security Requirements for Google Providers

This document describes the baseline security controls that web applications provided by Google partners must comply with. In order to prevent security issues, security controls must be carefully designed and should be regularly tested for their effectiveness.

 BARCLAYS

Supplier Control Obligation
(SCO)

Information and Cyber Security
(ICS)

Requisitos de seguridad: Ejemplos

Ejemplo incorrecto	Un ejemplo mejor
Las contraseñas deben ser seguras	Las contraseñas deberán tener al menos 12 caracteres de longitud y se rechazarán si se encuentran en una lista de 10.000 contraseñas comprometidas
Los datos estarán protegidos	Todos los PII (según la RGPD) estarán cifrados en reposo utilizando AES256-CTR. Las claves se almacenarán separadas de los datos que protegen.

Requisitos de seguridad: Ejemplos

Sujeto	Condición	Restricción	Verificación
¿A qué actor o componente se le aplica?	¿Bajo qué circunstancias?	¿Qué tiene que cumplirse? Específico y medible	¿Cómo se puede verificar su funcionamiento? (no se incluye siempre pero es deseable)

“La aplicación [sujeto], deberá, después de cinco intentos de inicio de sesión fallidos en el periodo de diez minutos, [condición] bloquear la cuenta durante treinta minutos y registrar el evento [restricción]. Se verifica mediante un test automatizado de secuencia de inicio de sesión con un script [verificación]”

¿Y con la IA?

- Si se lo pedimos a un LLM, nos dará una bonita ristra de requisitos que pareciera completa.
 - Probablemente no lo sea.

¿Y con la IA?

***Abstract*—Manifold approaches to security requirements engineering have been proposed, yet there is no consensus how to elicit, analyze, or express security needs. This perspective paper systematizes the problem space of security requirements engineering. Security needs result from the interplay of three dimensions: threats, security goals, and system design. Elementary statements can be made in each dimension, but such one-dimensional requirements remain partial and insufficient. To understand security needs, one has to analyze their interaction. Distinct analysis tasks arise for each pair of dimensions and are supported by different techniques: risk analysis, as in CORAS, between threats and security goals; security design, as exemplified by the framework of Haley et al., between goals and design; and security design analysis, such as Microsoft's threat modeling technique with data flow diagrams and STRIDE, between design and threats. All three perspectives are necessary to develop secure systems. Security requirements engineering must iterate through them, because threats determine the relevance of security goals, security design seeks ways to fulfill them, and design choices themselves influence threats and security goals.**

Türpe, Sven. "The trouble with security requirements."
In *2017 IEEE 25th International Requirements
Engineering Conference (RE)*, pp. 122-133. IEEE,
2017.

¿Y con la IA?

- Amenazas:
 - ¿Quiénes son los adversarios?
 - Capacidades, objetivos, contexto operacional
 - La IA generalmente produce categorías genéricas (inyección, bypass de autenticación...)
- Objetivos
 - Condiciones que se tienen que cumplir incluso bajo ataque
 - “Implementar control de acceso” no es un objetivo
 - “El usuario A no puede leer los registros del usuario B aún teniendo un token válido” sí puede serlo
- Diseño:
 - ¿Cómo se manifiestan las amenazas en ESTA arquitectura?
- La IA no tiene ningún conocimiento de la topología de despliegue, configuración de identidades, flujos de datos...
 - Proveer de ese contexto completo es difícil.

Cómo evitar vulnerabilidades

Diseño: Modelado de amenazas

Diseño seguro de software

- El diseño seguro de software es un campo amplio y complejo, por lo que daremos un par de pinceladas en dos aspectos:
 - Modelado de amenazas
 - Validación de inputs
- Y dejamos fuera otros aspectos como...
 - Principios de desarrollo de software seguro de Saltzer & Schroeder (¡pero muy recomendable!)

Modelado de amenazas

“El modelado de amenazas es un proceso por el cual se pueden identificar, y enumerar amenazas potenciales como vulnerabilidades estructurales o la falta de salvaguardas apropiadas, así como priorizar las mitigaciones.

tl;dr: encontrar debilidades y priorizarlas”

Abhijeth Dugginapeddi

Modelado de amenazas

- El framework de las 4 preguntas de Adam Shostack
 - ¿Sobre qué estamos trabajando?
 - ¿Qué puede ir mal?
 - ¿Qué vamos a hacer al respecto?
 - ¿Hemos realizado un buen trabajo?

Modelado de amenazas

- El framework de las 4 preguntas de Adam Shostack
 - ¿Sobre qué estamos trabajando? > ¿Qué es lo que queremos proteger?
 - ¿Qué puede ir mal? > ¿De qué o quién lo queremos proteger?
 - ¿Qué vamos a hacer al respecto? > ¿Qué estamos haciendo para protegerlo?
 - ¿Hemos realizado un buen trabajo? > Revisión de los resultados

Modelado de amenazas: STRIDE

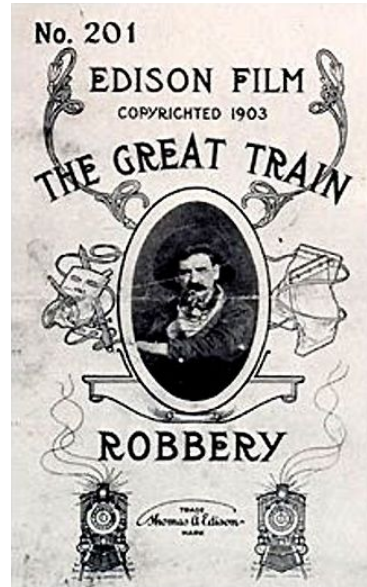
- Hay diferentes metodologías para la realización de este modelado
- STRIDE es una de las más populares
 - **S**poofing
 - Autenticación
 - **T**ampering
 - Integridad
 - **R**epudiation
 - No-repudio
 - **I**nformation Disclosure
 - Confidencialidad
 - **D**enial of Service
 - Disponibilidad
 - **E**levation of Privilege
 - Autorización

Modelado de amenazas: STRIDE

- Pasos:
 - Diagramas de los flujos de datos
 - Identificación/listado de amenazas
 - Mitigaciones
 - Validación
- Existen herramientas que partiendo de la arquitectura, listan las amenazas, pero (de momento) siempre se necesita intervención manual.

Modelado de amenazas: STRIDE

- Caso: El robo al tren
- Hay un tren cargado con un vagón cargado de dinero
- ¿Cómo robaríais **vosotros** el dinero?(*)



(*) Es un experimento mental, no pretendemos robar un tren de verdad

Modelado de amenazas: STRIDE

Nombre de la amenaza	Descripción	STRIDE	Ranking/ Probabilidad	Mitigación	Validación

Modelado de amenazas: STRIDE

- Pasos:
- Entender el caso
 - Alto nivel
 - Arquitectura, flujos de datos etc.
 - A nivel de detalle
 - Qué
 - Quién
 - Cómo
- Identificar las posibles amenazas
 - Éste es un esfuerzo colaborativo. La comunicación entre los ingenieros de software y los responsables del modelado de amenazas es **vital**.
- Existen herramientas que partiendo de la arquitectura, listan las amenazas, pero (de momento) siempre se necesita intervención manual.

Modelado de amenazas: STRIDE

- Pasos:
- Entender el caso
 - Alto nivel
 - Arquitectura, flujos de datos etc.
 - A nivel de detalle
 - Qué
 - Quién
 - Cómo
- Identificar las posibles amenazas
 - Éste es un esfuerzo colaborativo. La comunicación entre los ingenieros de software y los responsables del modelado de amenazas es **vital**.
- Existen herramientas que partiendo de la arquitectura, listan las amenazas, pero (de momento) siempre se necesita intervención manual.

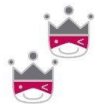
Modelado de amenazas: STRIDE

- Identificar las posibles amenazas, requiere cierto conocimiento experto
- Existen herramientas que nos ayudan a pensar.
 - Por ejemplo, juegos de cartas

2

Spoofing

An attacker could squat on the random port or socket that the server normally uses.

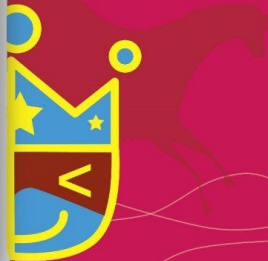


2

J

Spoofing

An attacker could steal credentials stored on the client and reuse them.



3

Tampering

An attacker can take advantage of your custom key exchange or integrity control which you built instead of using standard crypto.



3

K

Tampering

An attacker can load code inside your process via an extension point.



7

Information Disclosure

An attacker can act as a "man in the middle" because you don't authenticate endpoints of a network connection.



7

6

Denial of Service

An attacker can make a server unavailable or unusable without ever authenticating, but the problem goes away when the attacker stops (**server, anonymous, temporary**).



6

Q

Repudiation

An attacker can say "I didn't do that," and you would have no way to prove them wrong.



K

Elevation of Privilege

An attacker can inject a command that the system will run at a higher privilege level.



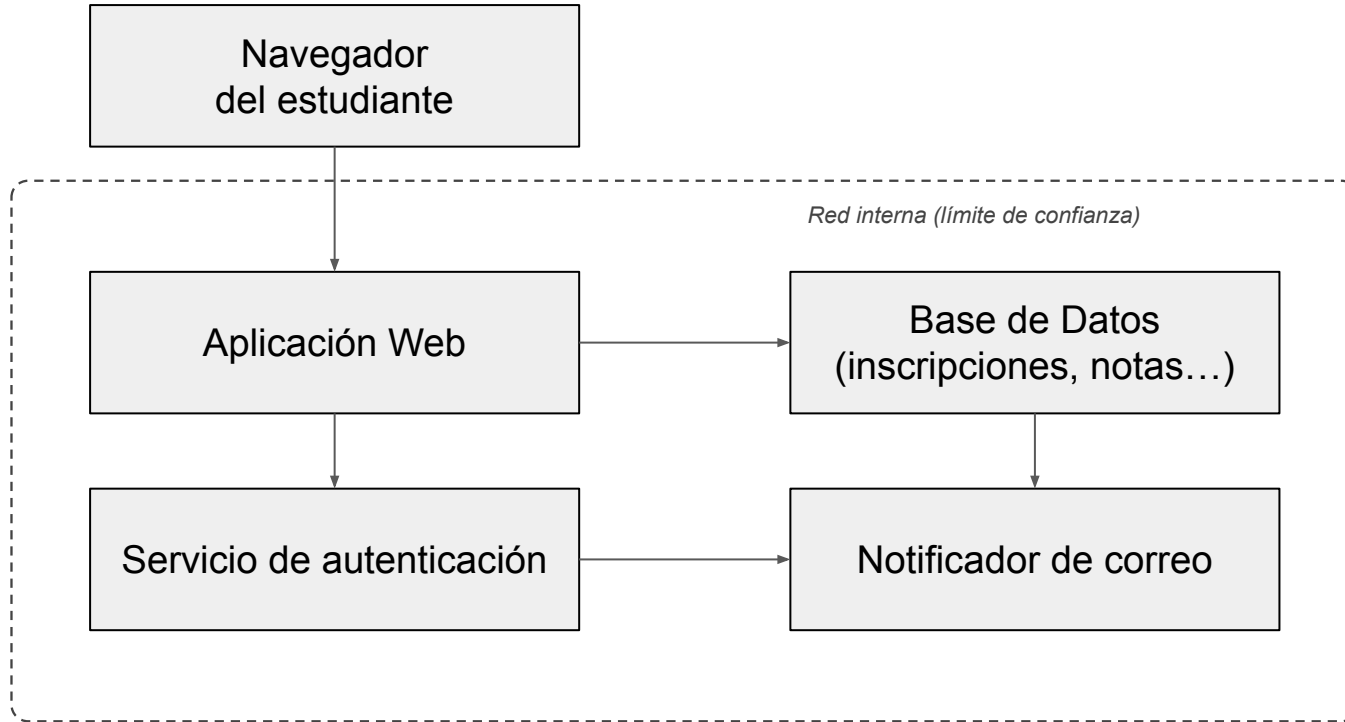
Modelado de amenazas: Elevation of privilege

- [Juego de cartas](#) creado por Adam Shostack para estimular la creatividad en el proceso de modelado de amenazas
- Existen diferentes variantes: [online](#) y físico
- Juegos similares:
 - [OWASP Cornucopia](#): Aplicaciones web
 - [OWASP Cumulus](#): Orientado a cloud y devops
 - [Elevation of Privacy](#): Expansión orientada a la privacidad
 - [Elevation of MLSec](#): Expansion orientada al ML/AI

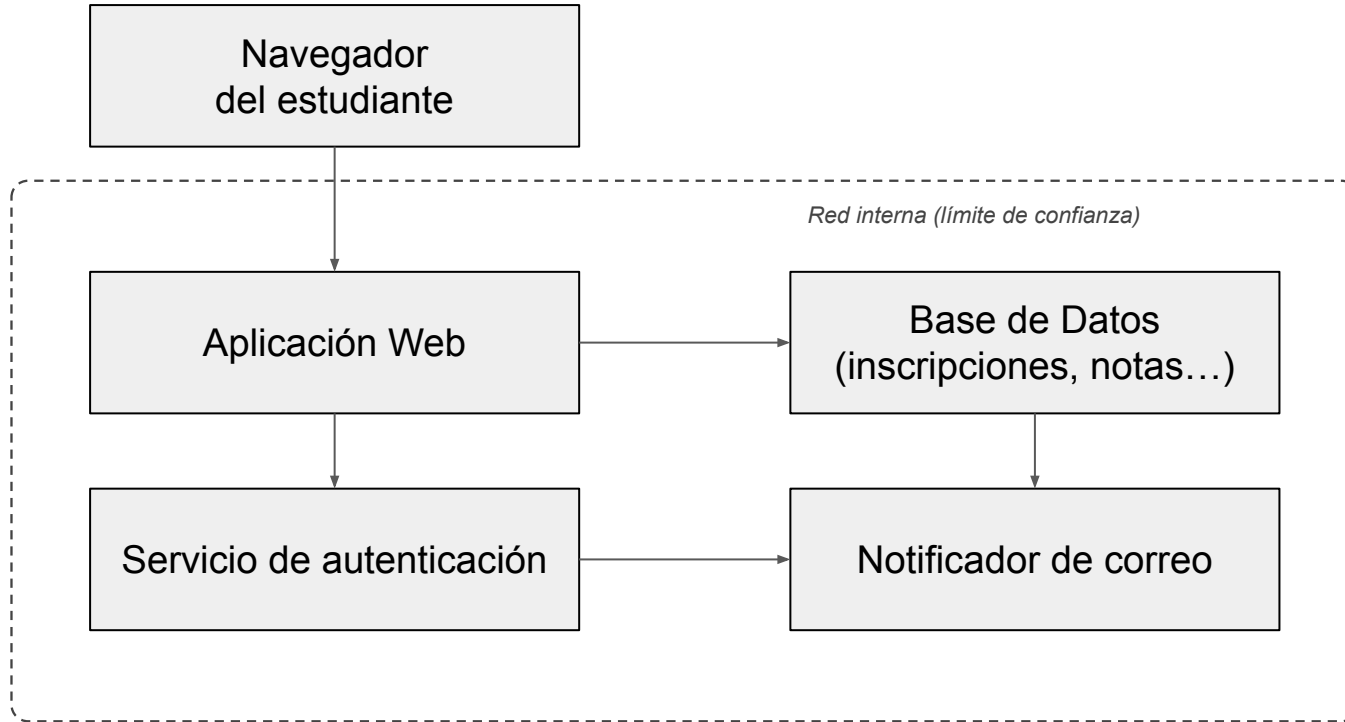
“Elevation of privilege versión Mikel”



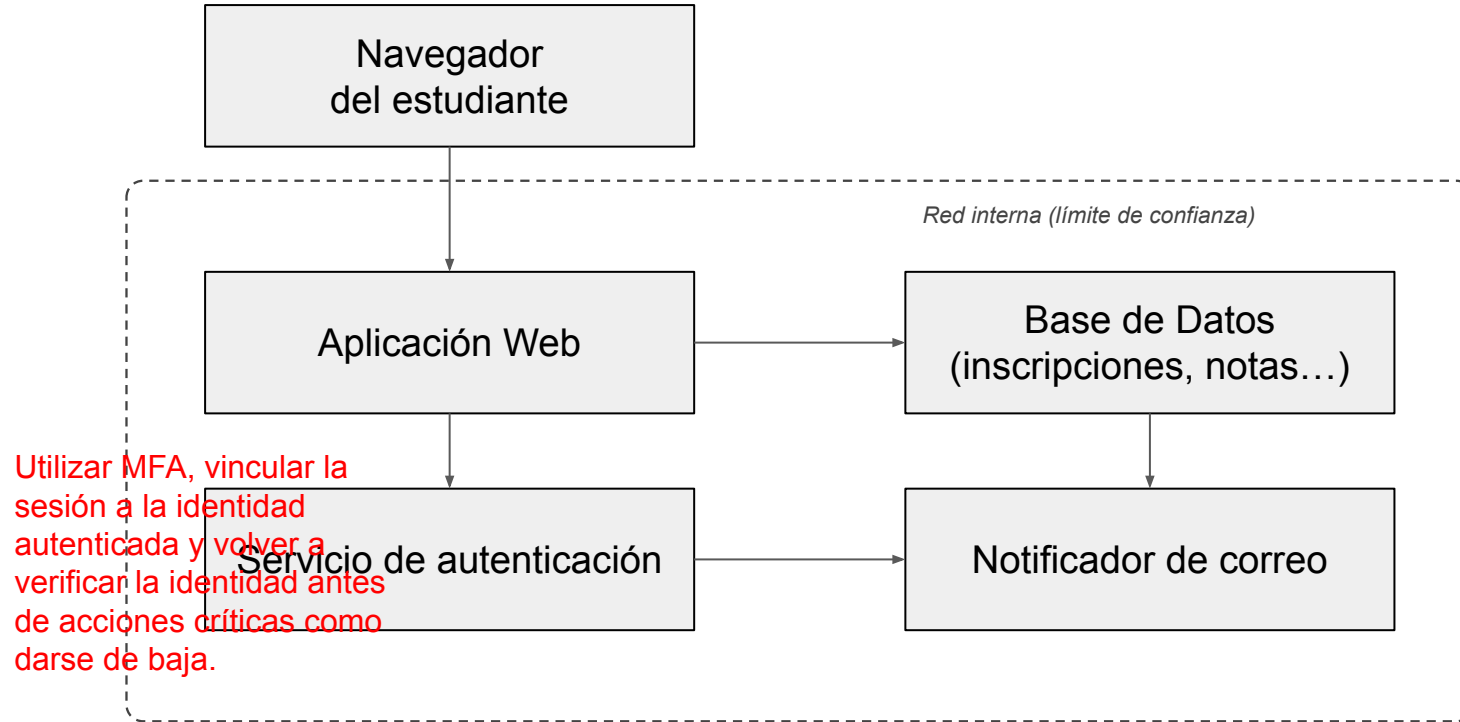
“Elevation of privilege versión Mikel”



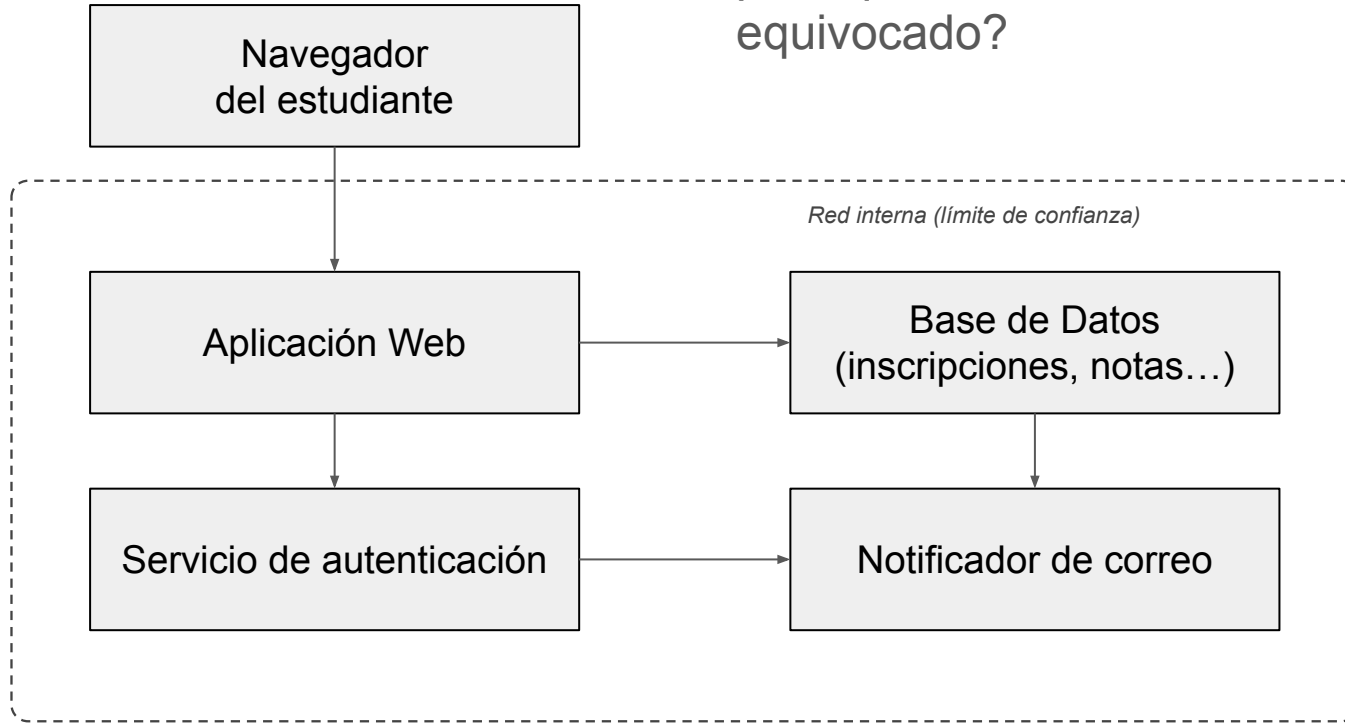
¿Podría alguien suplantar a un estudiante para inscribirlo o darlo de baja de un curso en su nombre?



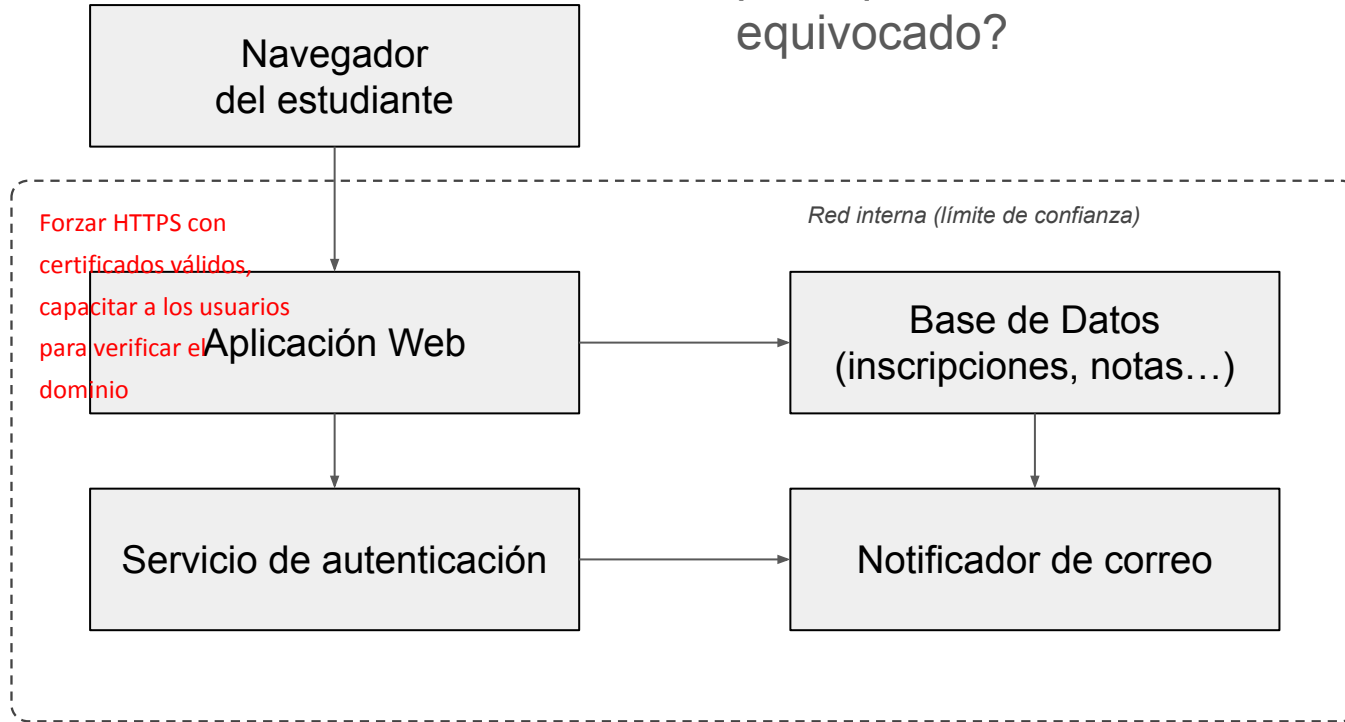
¿Podría alguien suplantar a un estudiante para inscribirlo o darlo de baja de un curso en su nombre?



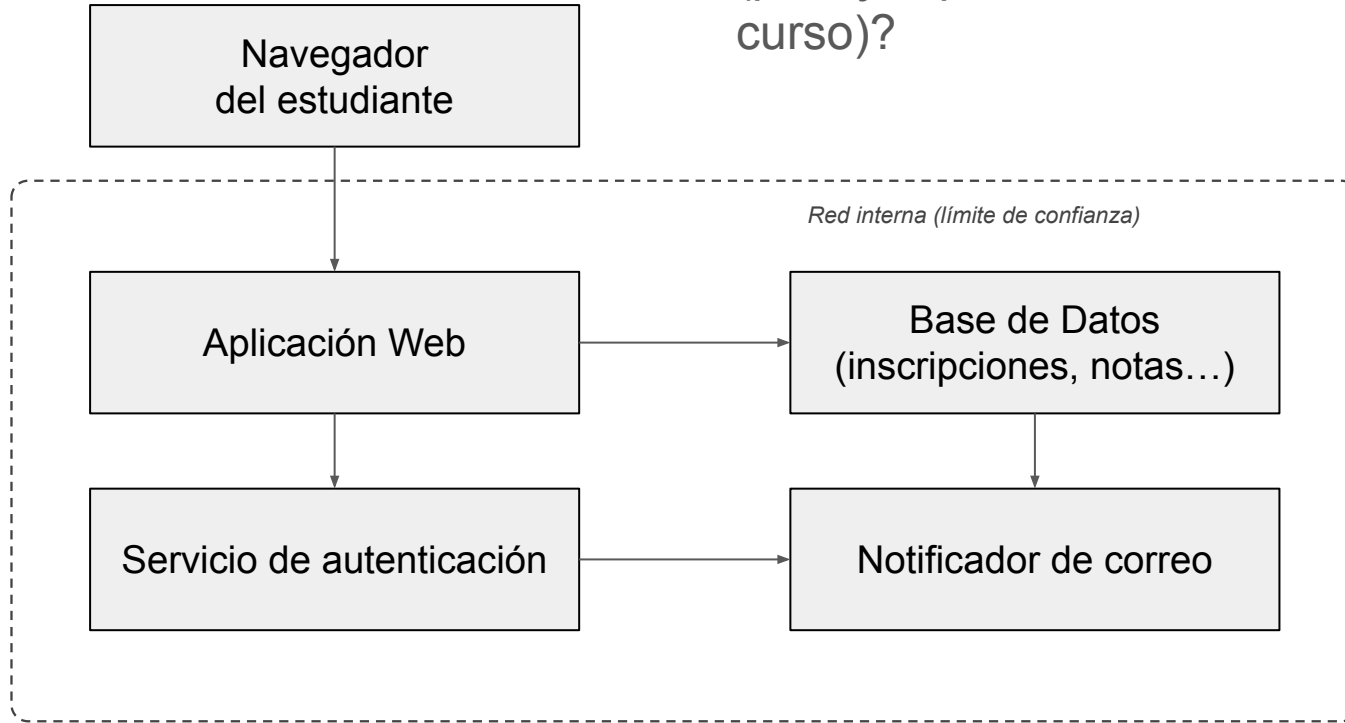
¿Podría una versión falsa de la aplicación web engañar al navegador del estudiante para que envíe sus credenciales al lugar equivocado?



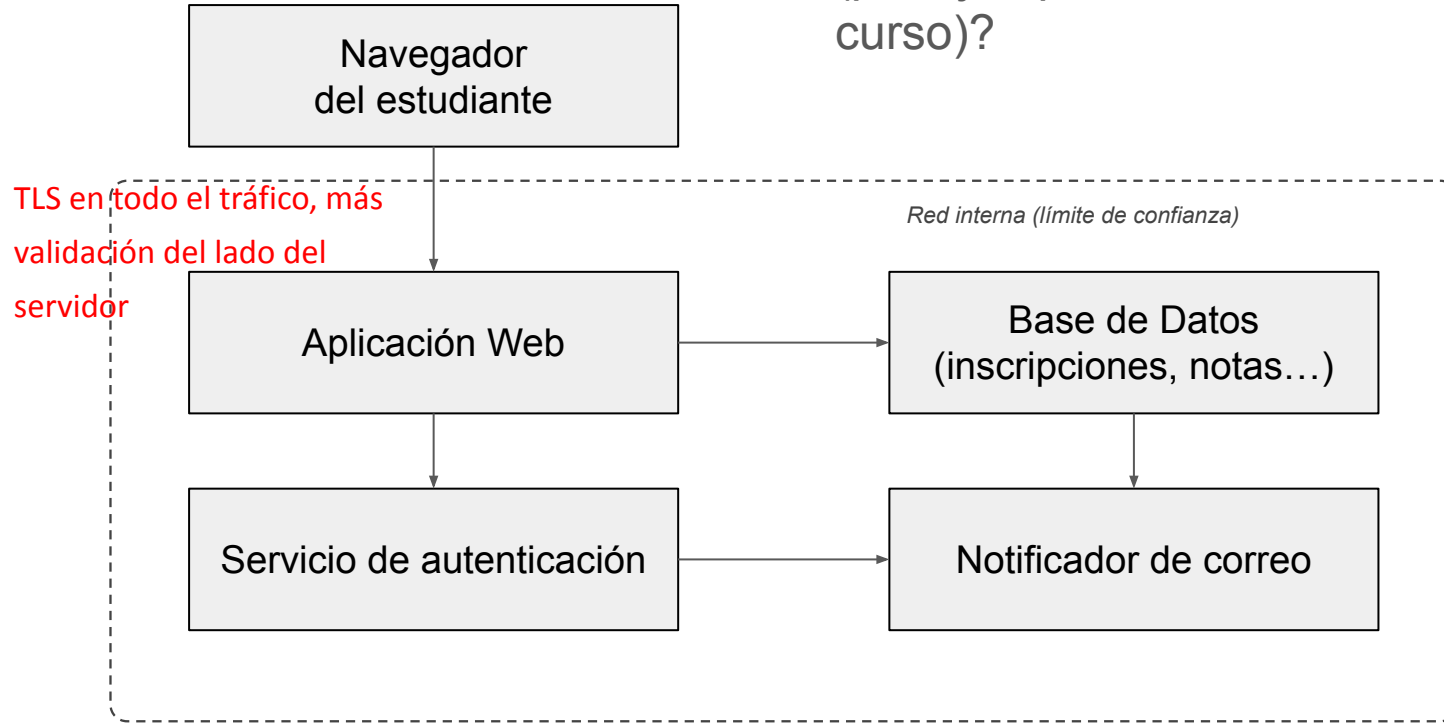
¿Podría una versión falsa de la aplicación web engañar al navegador del estudiante para que envíe sus credenciales al lugar equivocado?



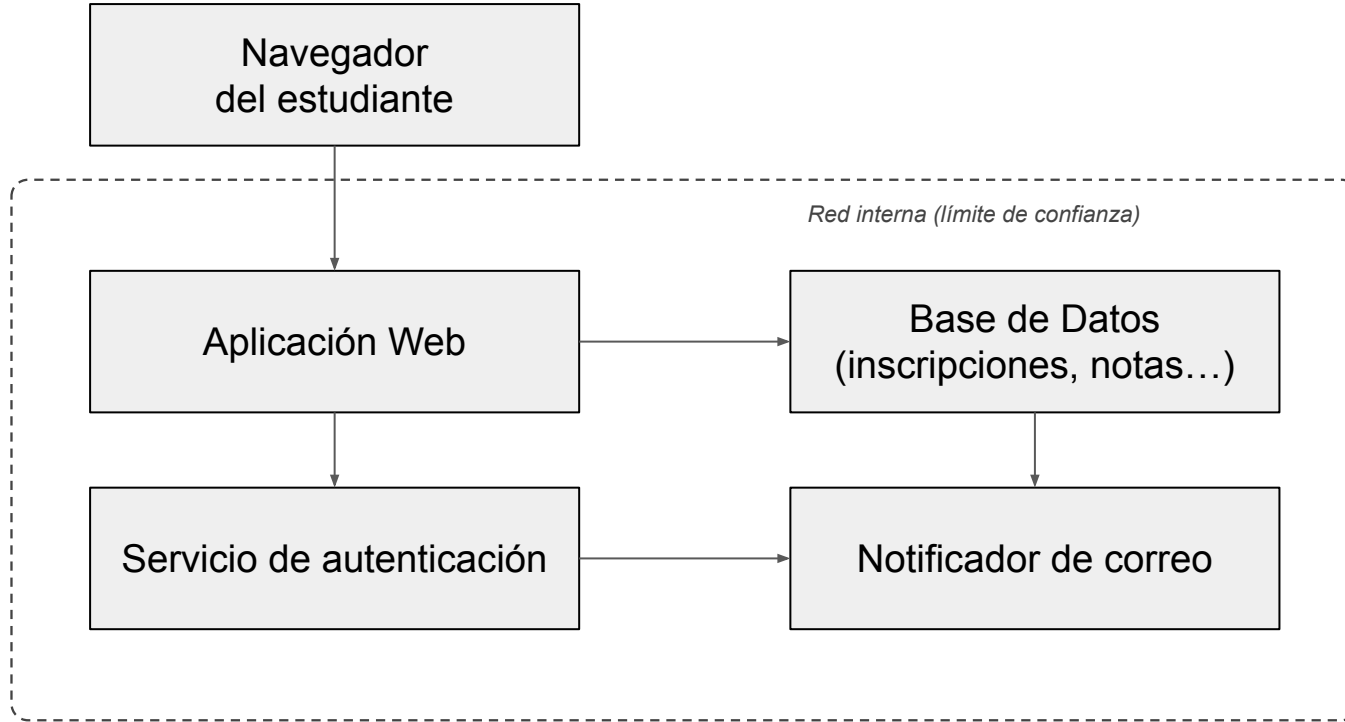
¿Podría interceptarse y modificarse una solicitud entre el navegador y la webapp (por ejemplo, cambiando el ID de un curso)?



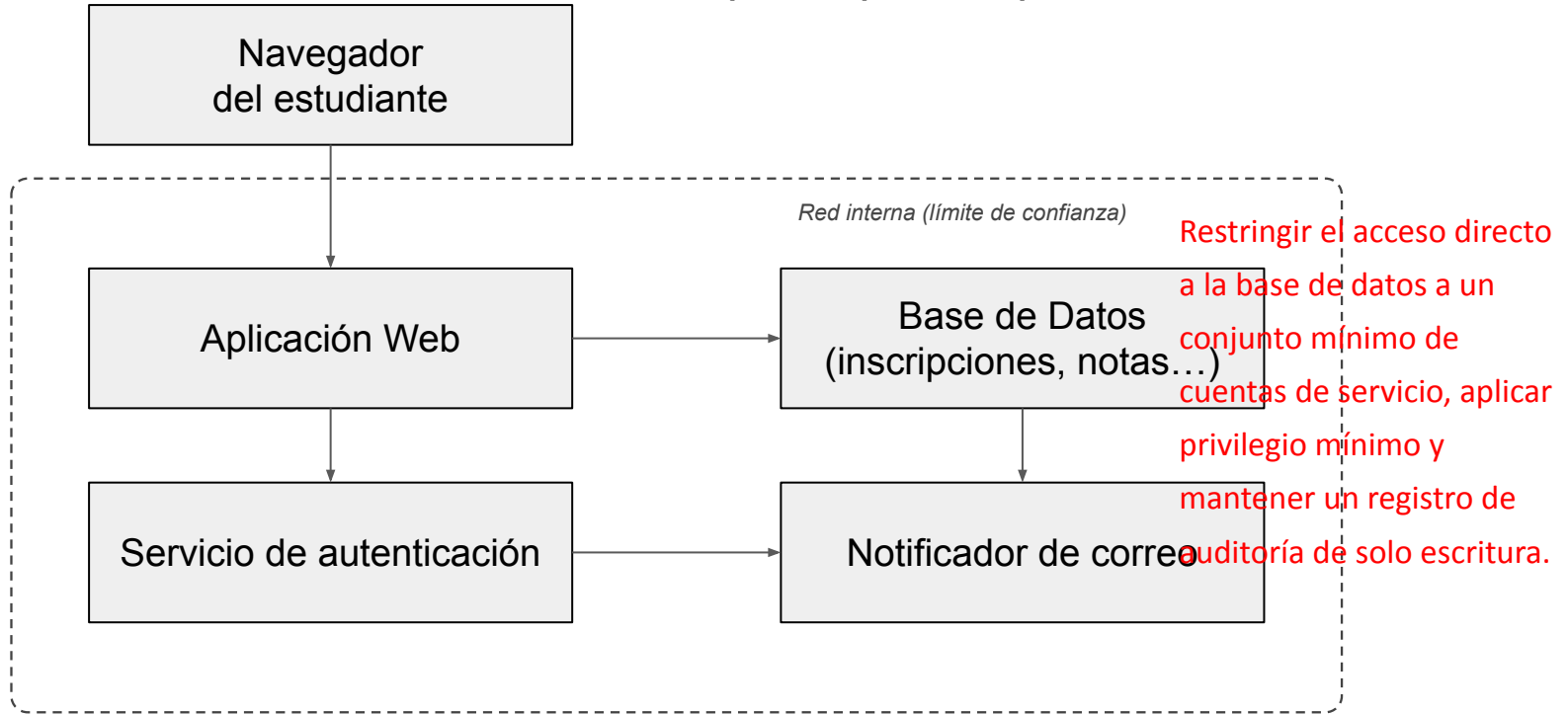
¿Podría interceptarse y modificarse una solicitud entre el navegador y la webapp (por ejemplo, cambiando el ID de un curso)?



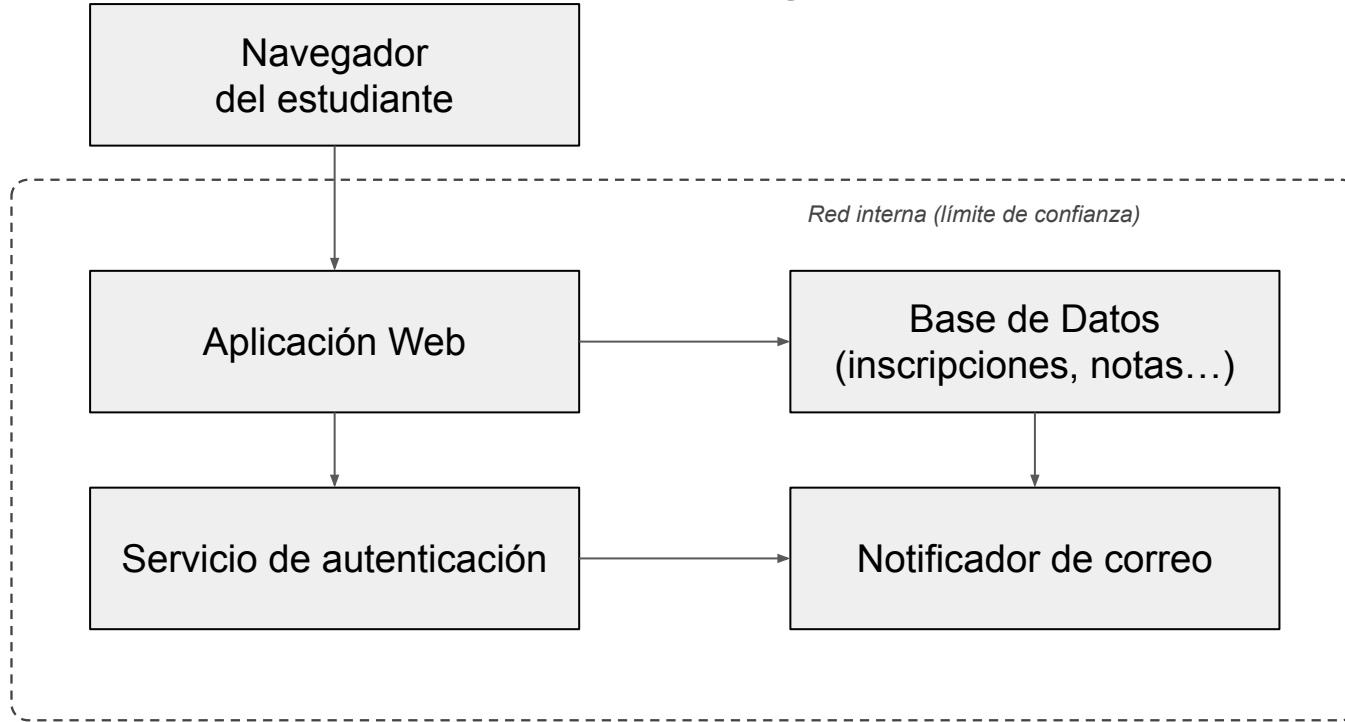
¿Podría alguien con acceso a la base de datos alterar notas o inscripciones sin pasar por la aplicación?



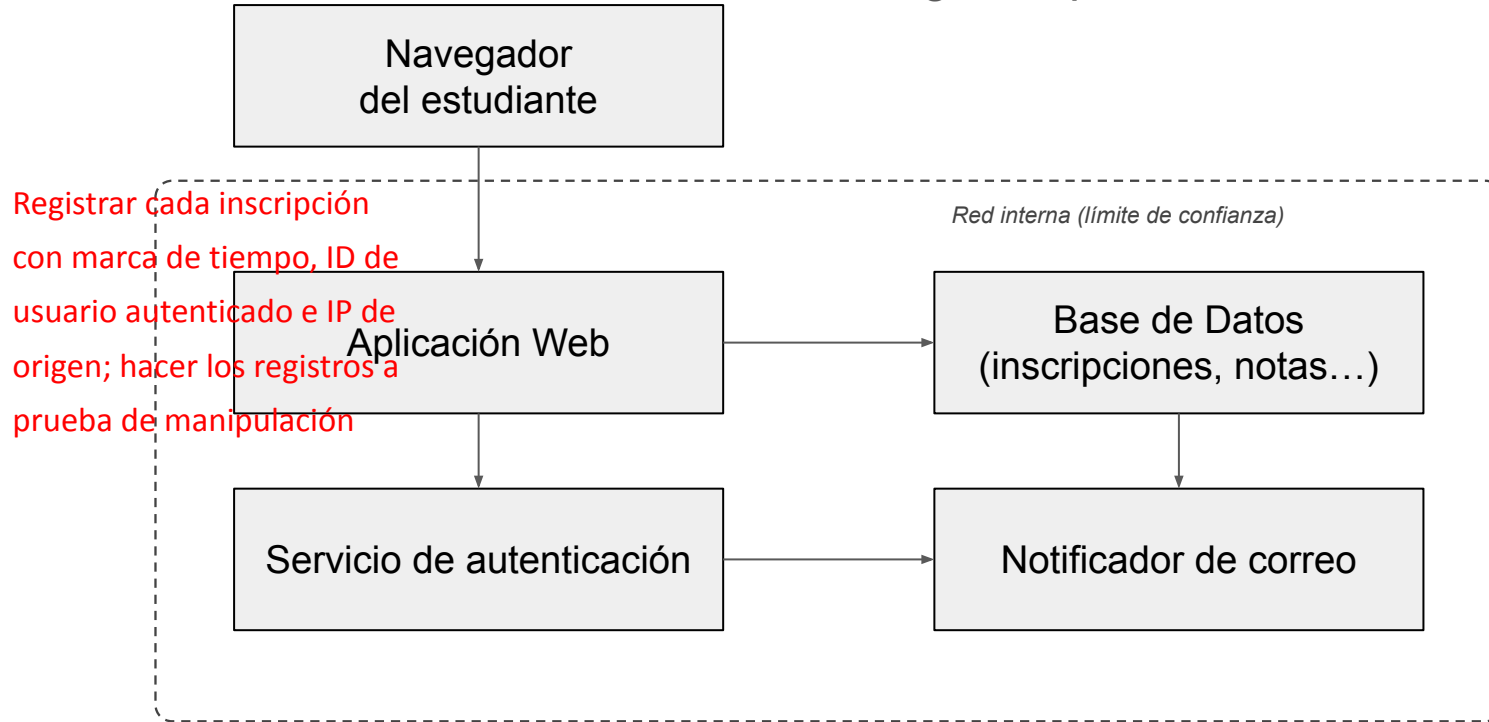
¿Podría alguien con acceso a la base de datos alterar notas o inscripciones sin pasar por la aplicación?



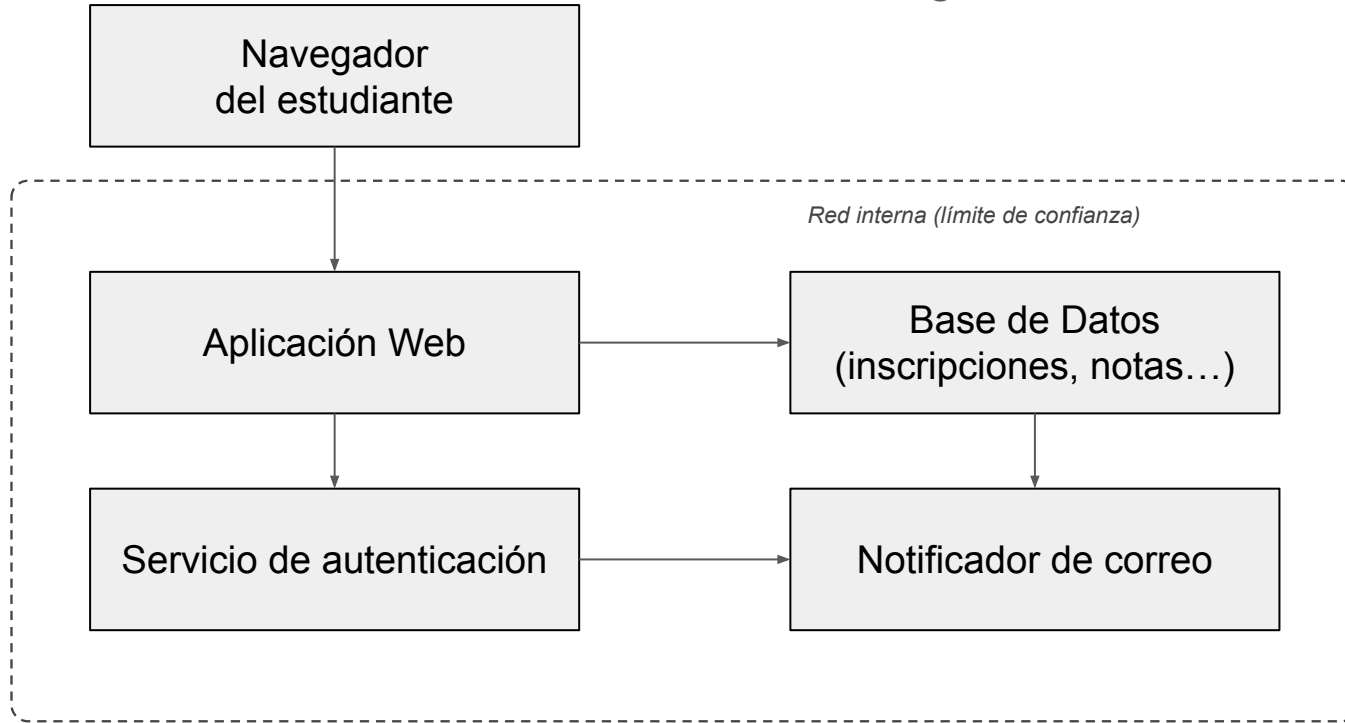
Si un estudiante afirma "nunca me inscribí en ese curso", ¿tiene el sistema un registro que lo demuestre?



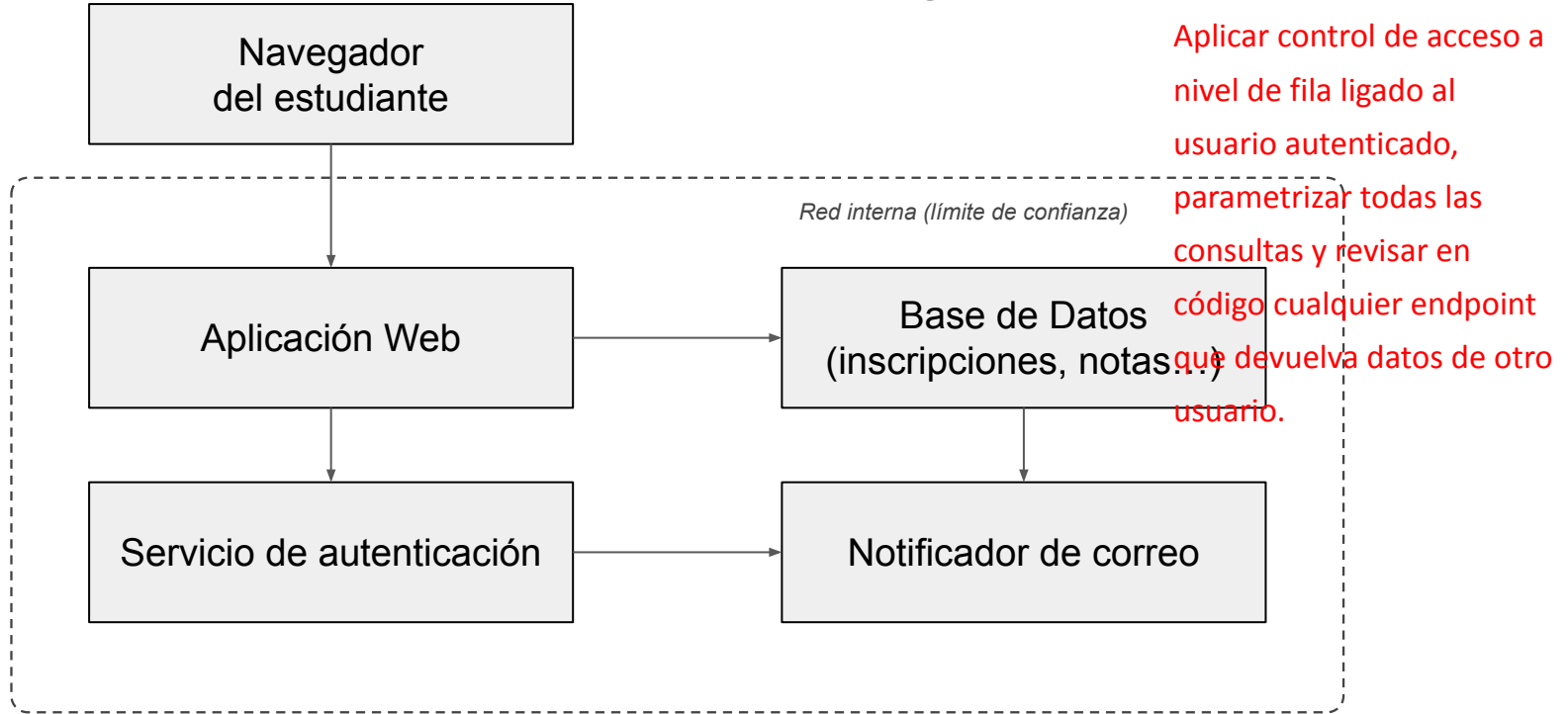
Si un estudiante afirma "nunca me inscribí en ese curso", ¿tiene el sistema un registro que lo demuestre?



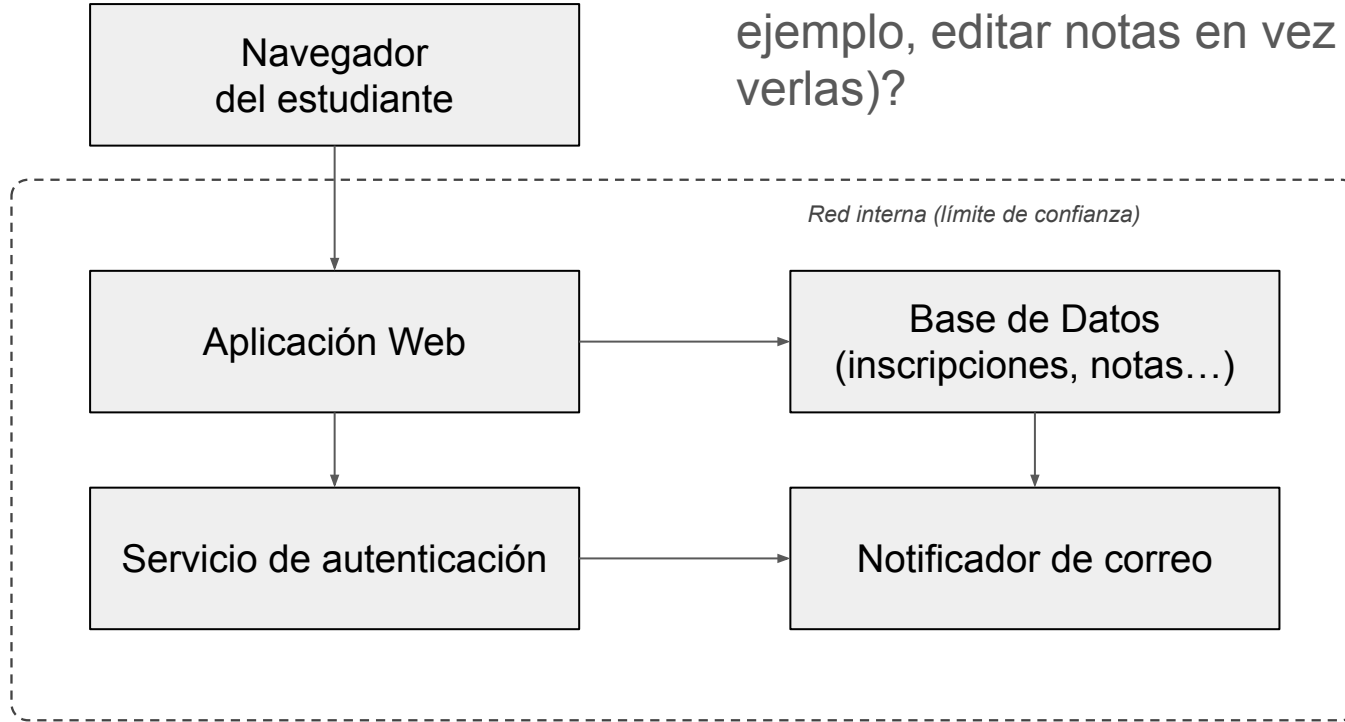
¿Podría la base de datos filtrar las notas de un estudiante a otro por una consulta mal configurada?



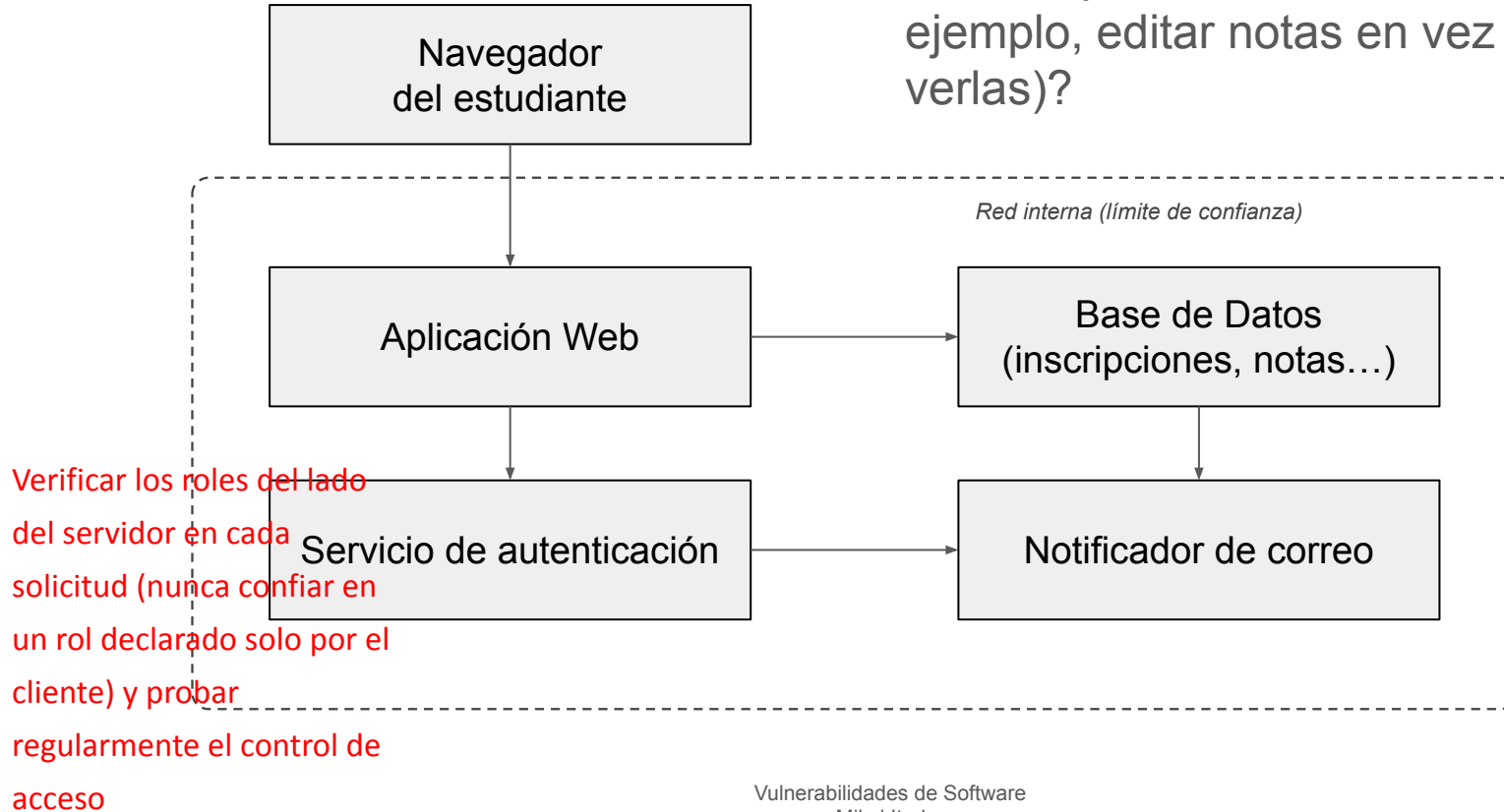
¿Podría la base de datos filtrar las notas de un estudiante a otro por una consulta mal configurada?



¿Podría un estudiante explotar el servicio de autenticación para obtener permisos de nivel personal administrativo (por ejemplo, editar notas en vez de solo verlas)?



¿Podría un estudiante explotar el servicio de autenticación para obtener permisos de nivel personal administrativo (por ejemplo, editar notas en vez de solo verlas)?



Resultado: tabla STRIDE

Nombre de la amenaza	Descripción	STRIDE	Prioridad	Mitigación	Validación
S.01	Suplantar a un estudiante para inscribirlo o darlo de baja de un curso en su nombre	S	Alta	Utilizar MFA, vincular la sesión a la identidad autenticada y volver a verificar la identidad antes de acciones críticas como darse de baja.	Comprobación existencia MFA y vinculación. Test de penetración
...	...	...	...	...	...

¿Y con la IA?

Las herramientas IA nos dan tablas STRIDE al momento. Sistemáticamente incompletas.

¿Y con la IA?

- Varias razones (falta de contexto, sin modelar al adversario...). Para mi la fundamental:
- No hay discusión. STRIDE y el modelado de amenazas debería ser colectivo. La IA únicamente genera un documento.

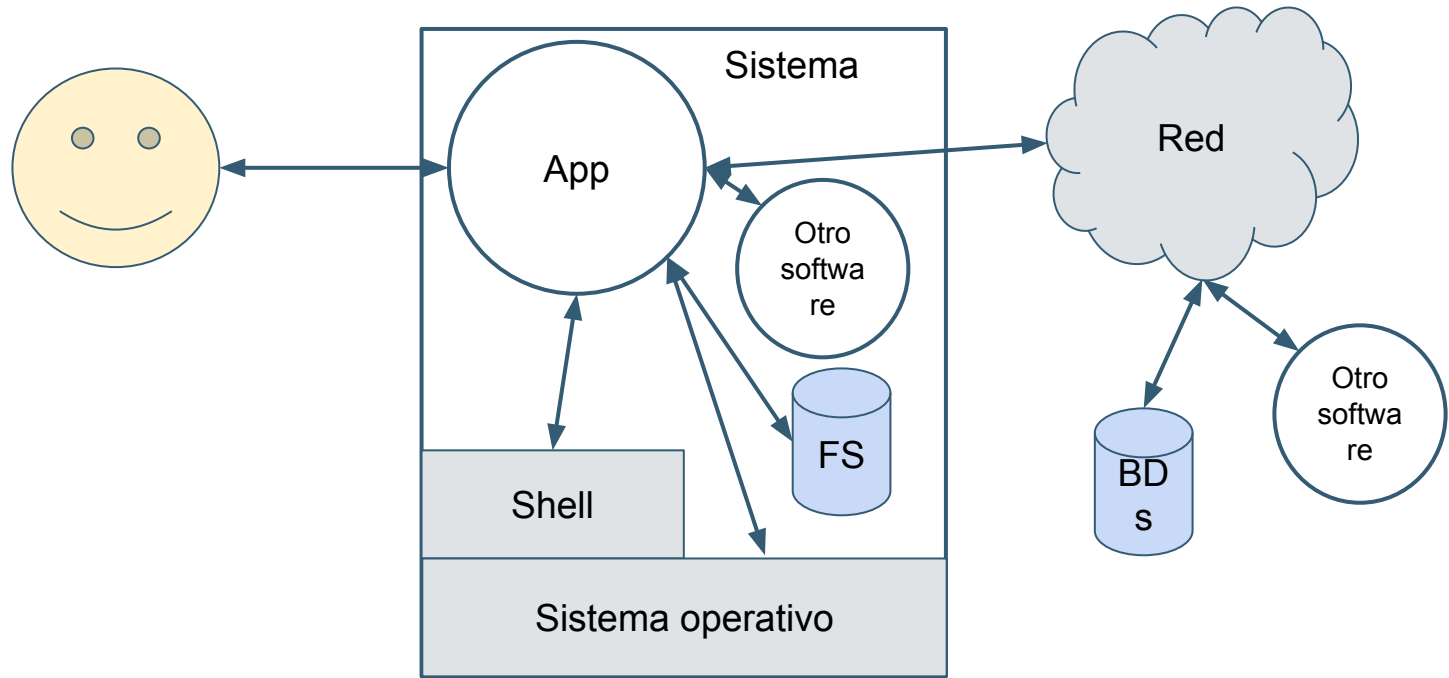
Cómo evitar vulnerabilidades

Diseño: Validación de entradas

Validación de entradas

- La validación incorrecta de entradas es una de las causas más comunes para vulnerabilidades
 - Es necesario validar todos los inputs
 - Cuatro pasos:
1. Identificar todas las fuentes de entrada. ¿Qué fuentes puede tener una aplicación?

Validación de entradas



Validación de entradas

2. Los datos de todas las fuentes no confiables tiene que ser especificada y validada contra la especificación
 - a. La implementación debe estar diseñada para trabajar con cualquier combinación o rango de datos válidos
 - b. La naturaleza de los datos válidos se especifica en el diseño de la aplicación
3. Las especificaciones deben definir:
 - a. límites, valores max/min, longitud max/min, contenido válido, requisitos de inicialización así como requisitos de cifrado en tránsito o en almacenamiento en caso de ser necesario
4. Se debe asegurar que todas las entradas cumplen con las especificaciones (en la implementación)
 - a. Se pueden encapsular en una clase
 - b. Se deben validar las entradas lo antes posibles
 - c. Si la especificación no se cumple, se debe mostrar un mensaje de error explicativo

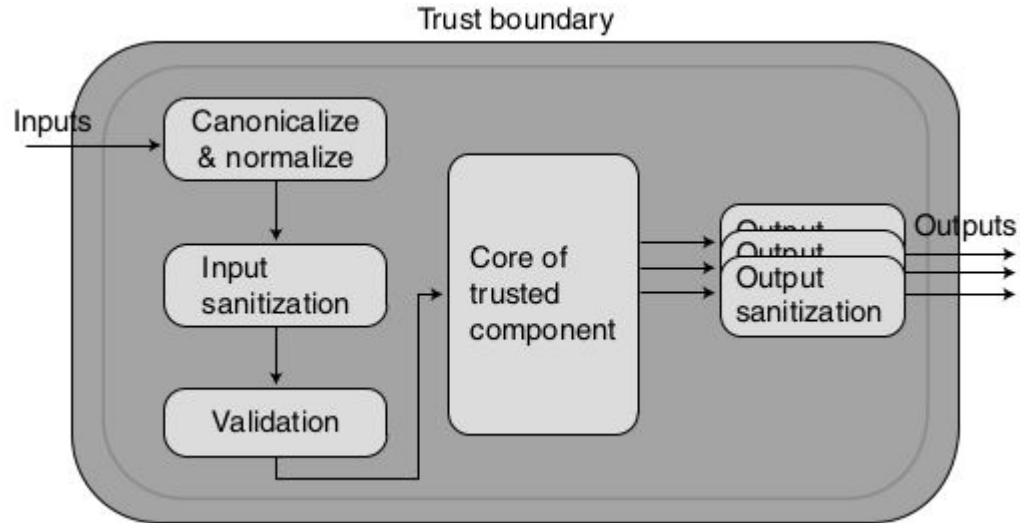
Validación de entradas

- Usad componentes estándares para la validación de entradas. Existen librerías y frameworks (en el caso de web)
- Si utilizáis alguna estructura de datos propia, podéis definir vuestra especificaciones utilizando librerías como Cerberus o Pydantic en Python
- Allowlist vs. Denylist

Validación de entradas

- El software dispone muchas veces de componentes y librerías múltiples.
- Cada componente puede operar un o más dominios de confianza, que los determina la arquitectura del sistema, la política de seguridad, los recursos necesarios y la funcionalidad.
- Por ello, establecemos, límites de confianza entre los componentes que corresponden a diferentes dominios de confianza
 - Cuanto más control tengamos del componente, más confianza podremos tener.
 - Ejemplos de límites de confianza:
 - Nuestra aplicación || Librería externa
 - Aplicación web en el servidor web de la DMZ || BD interna
 - Dispositivo IoT || Backend

Validación de entradas



Validación de entradas

- Hay diferentes pasos para asegurar que los datos que traspasan límites son apropiados y no maliciosos:
 - Canonicalización y normalización
 - Sanitización
 - Validación

Validación de entradas

- Canonicalización es el proceso de convertir un input a su forma inequívoca y no ambigua (forma canónica)
- La normalización es equivalente, pero puede darse pérdida de información (en la canonicalización la información se mantiene).
- Ejemplos:
 - Nombres de fichero: ~/Documents/photo.jpg > /home/miturbe/Documents/photo.jpg
 - Strings: Algunos caracteres UTF pueden ser decodificados de forma diferente en bytes:
 - é = U+0065 U+0301 (no-canónica)
 - é = U+00E9 (canónica)
 - URLs: http://example.com/page?query > http://example.com/page
- Ejemplo de normalización:
 - ~/Documents/photo.jpg > photo.jpg
- La normalización y canonicalización siempre se aplican antes de la validación.
 - Previene la explotación de la rutina de validación representando los datos de la forma válida

Validación de entradas

- La sanitización es la práctica de verificar que los datos de entrada se corresponden con los requisitos de nuestro componente
- Frecuentemente incluye la eliminación de caracteres no deseados, quitándolos, codificándolos o reemplazándolos de la entrada.
- La sanitización puede ocurrir antes o después de que los datos traspasen un límite de confianza (sanitización de inputs y outputs)
- Ejemplos:
 - “This is a string!!” > “this_is_a_string”
 - “1 OR 1=1;” > “1_OR_11”

Validación de entradas

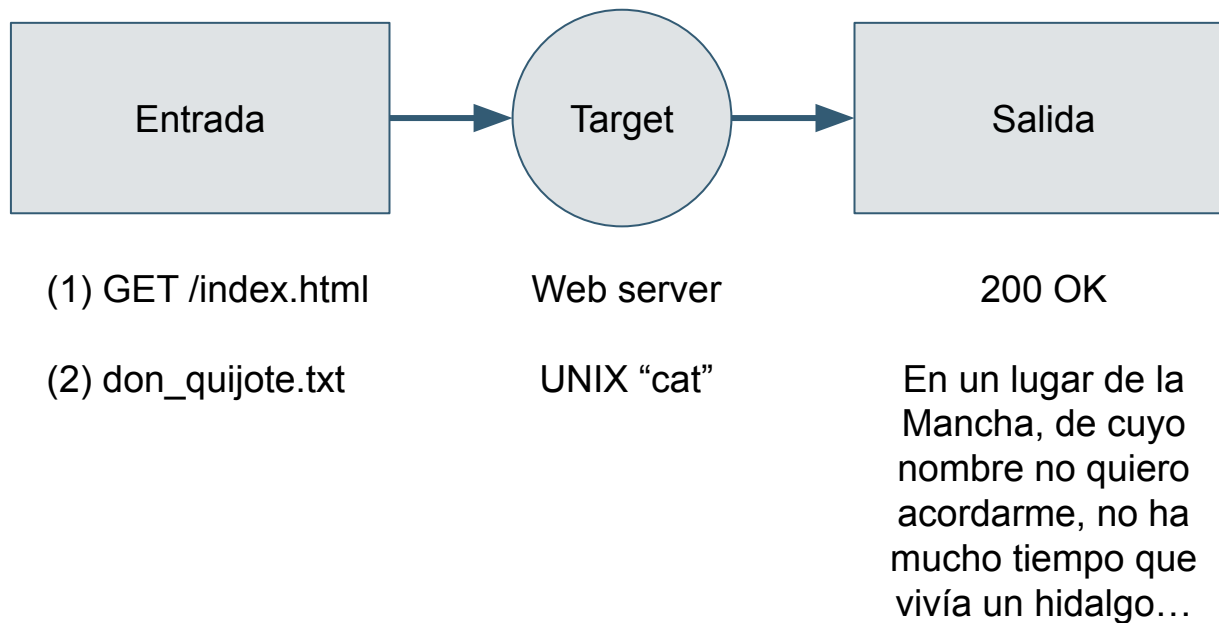
- Hay dos formas de abordar la sanitización:
 - Denylist:
 - Se definen explícitamente una serie de caracteres no permitidos
 - Los caracteres no permitidos se eliminan
 - Se deben identificar todos los inputs no-legítimos
 - Allowlist
 - Se identifican todos los caracteres permitidos en el sistema, definiendo como no permitidos el resto.
 - Forma de actuar más segura.

Validación de entradas

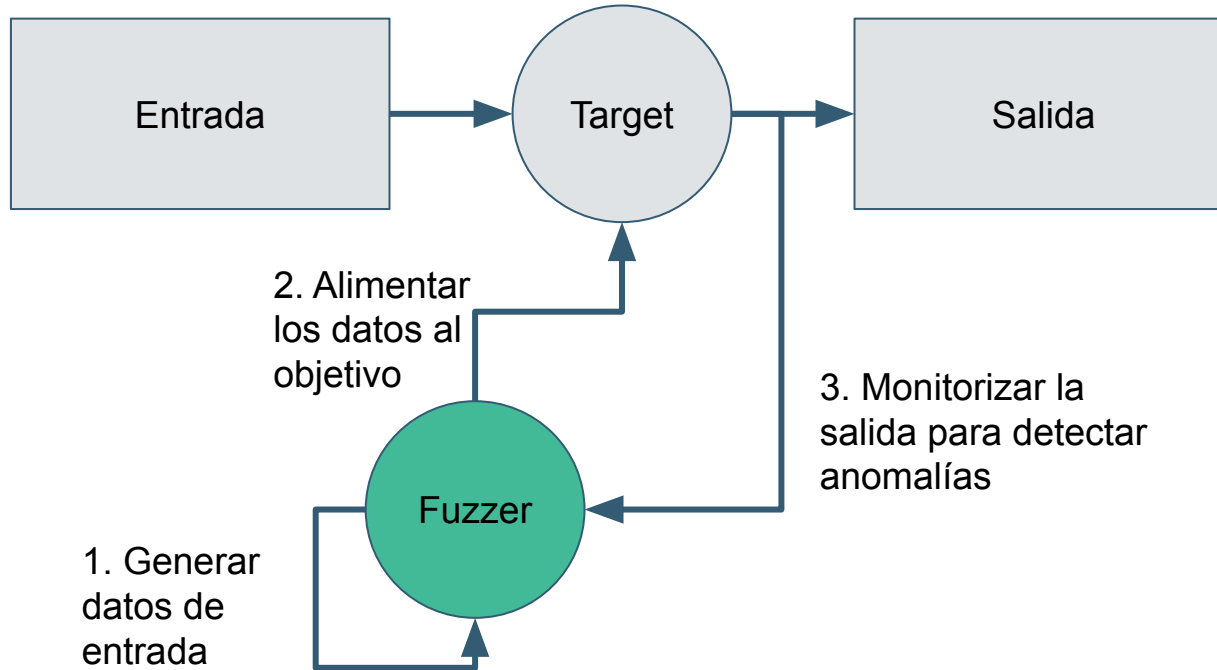
- La validación se encarga que los datos del input están dentro del dominio de lo que se consideran datos válidos.
- Ejemplos:
 - Edad = -5 > Edad = 0
 - Fecha de vuelta de viaje anterior a la fecha de salida

Búsqueda de vulnerabilidades

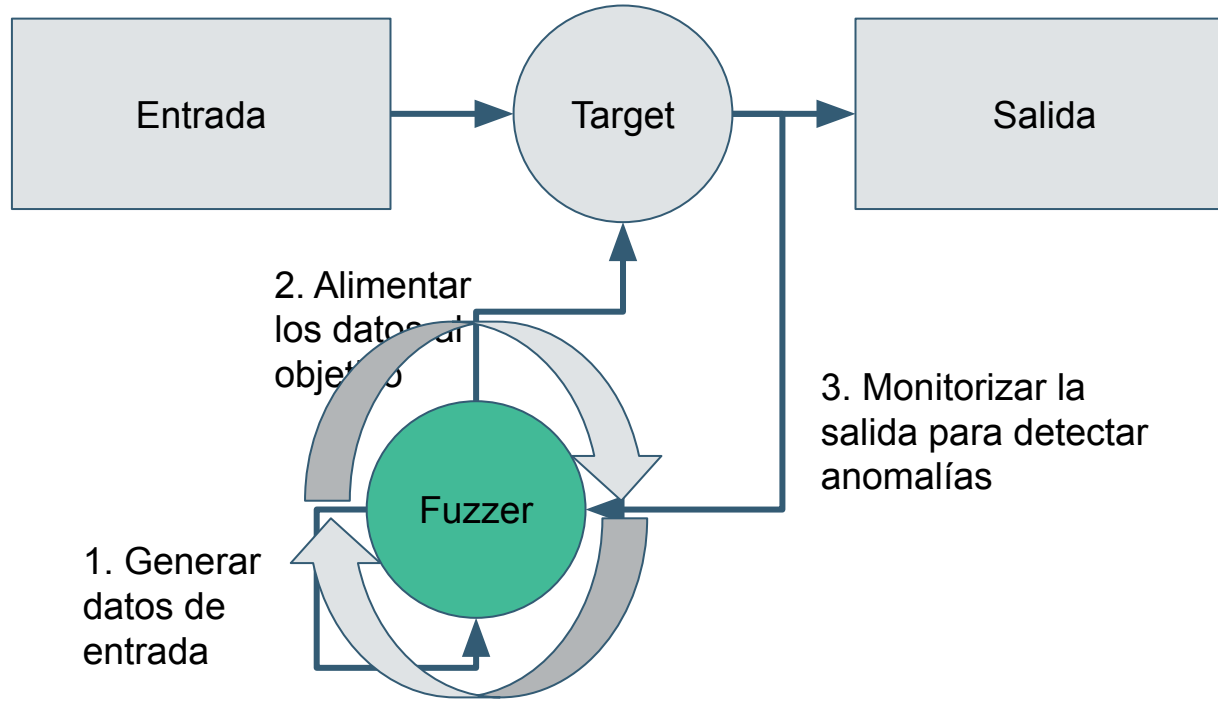
Fuzzing



Fuzzing



Fuzzing



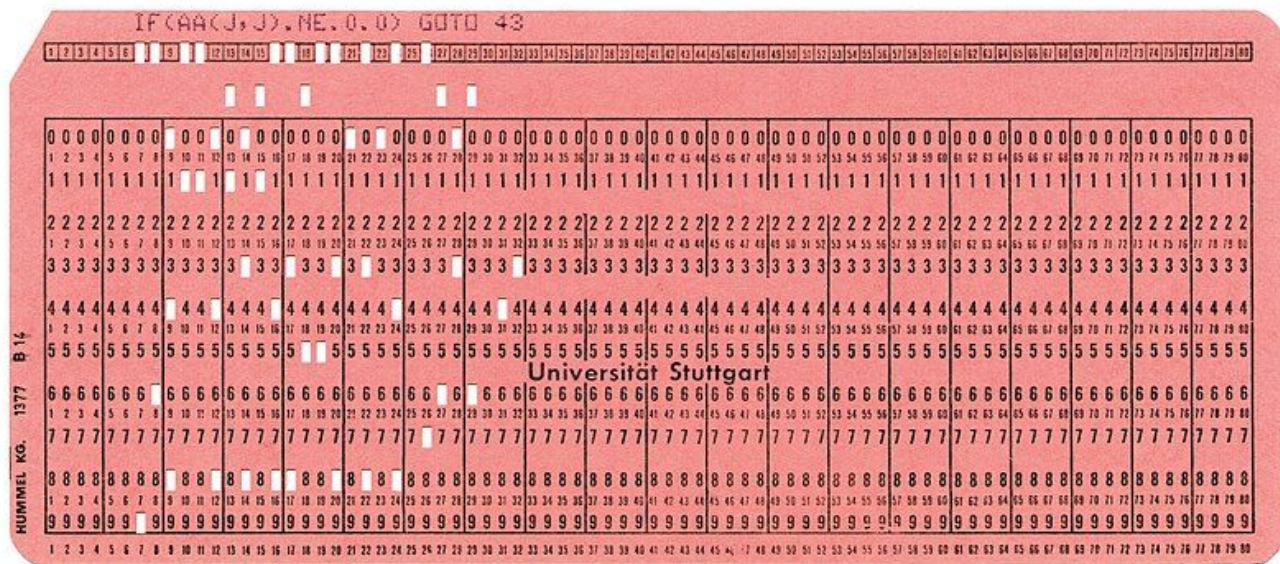
Fuzzing

- Equivalente a chocar miles de coches, de miles formas diferentes, para ver como actúan los sistemas de seguridad



Fuzzing

- Y aunque se utilice mucho y esté bastante en boga, no es algo nuevo



Fuzzing

- A finales de los 80 Barton Miller de Wisconsin University estaba conectado remotamente desde casa a la universidad utilizando una línea de 1200 baudios (*).
 - Debido al mal tiempo, la conexión se perdía
- Inspirado por ello, creó [una práctica](#) para sus alumnos.
- Enunciado original:
 - The Fuzz Generator: The goal of this project is to evaluate the robustness of various UNIX utility programs, given an unpredictable input stream. This project has two parts. First, you will build a fuzz generator. This is a program that will output a random character stream. Second, you will take the fuzz generator and use it to attack as many UNIX utilities as possible, with the goal of trying to break them. For the utilities that break, you will try to determine what type of input cause the break.

Fuzzing

- Resultados ([publicados en 1990](#))
 - we have been able to crash 25-33% of the utility programs on any version of UNIX that was tested
- Desde entonces, es el sistema más eficiente para la búsqueda de vulnerabilidades
- Objetivo: fuzzear nuestras aplicaciones para encontrar y parchearlas, antes de que las hagan los otros.
- ¿Y qué hacemos con la validación de inputs a la hora de fuzzear?

Quiz

wooclap.com

Código:

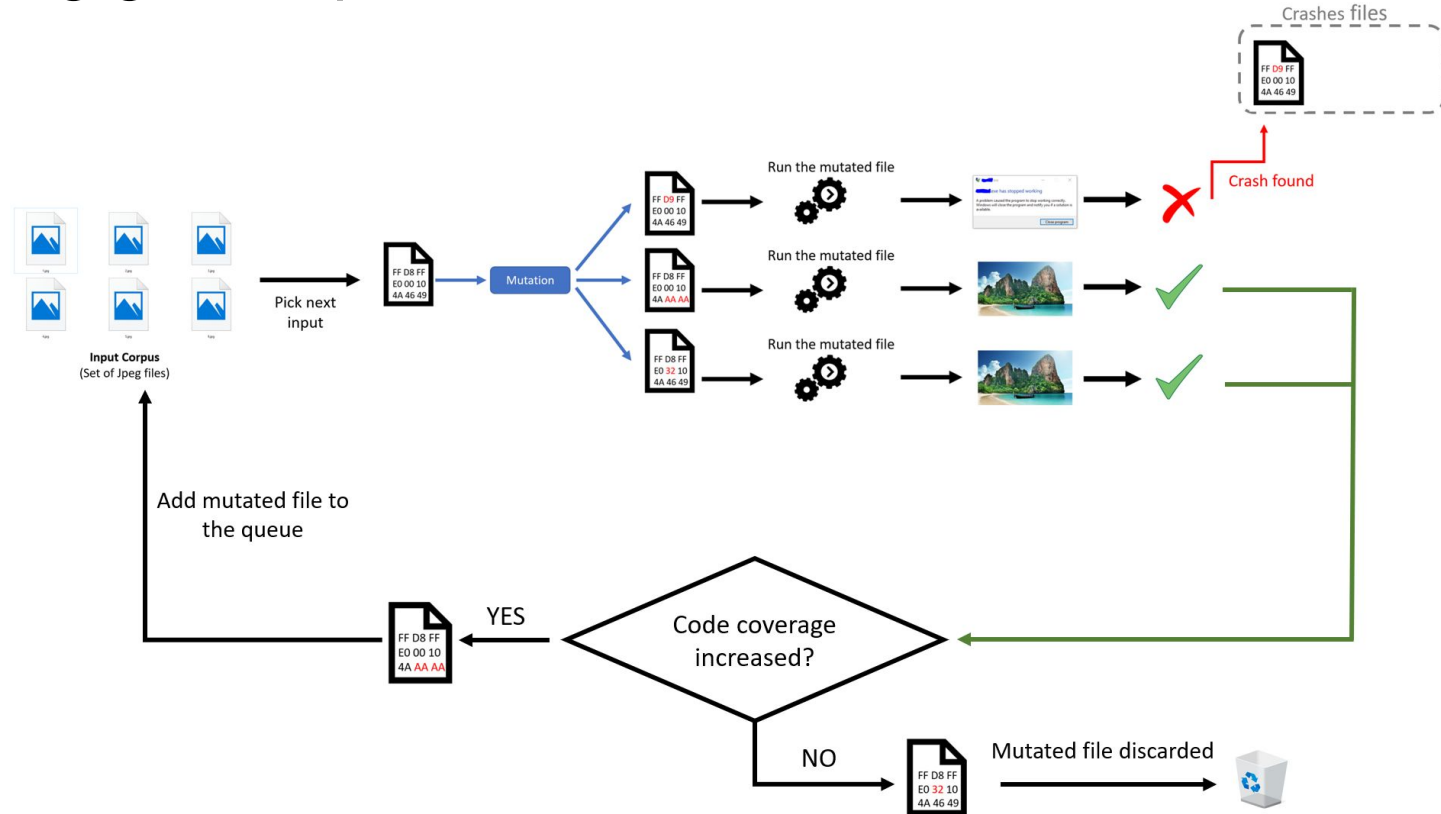
UXPJDA



Fuzzing

- No vamos a entrar a fondo en diferentes tipos de fuzzer existentes.
- Vamos a centrarnos en un tipo de fuzzer, AFL++
 - fuzzer evolutivo guiado por cobertura
 - guiado por cobertura: tiene como objetivo ampliar la cobertura del código, es decir, la extensión de las funcionalidades del programa testeado
 - Evolutivo: Partiendo de un tipo de entrada en concreto, muta sobre ellas para generar nuevas entradas.
 - Instrumenta los binarios (o los ejecuta sobre una plataforma de emulación) para tener visibilidad del proceso de fuzzing

Fuzzing guiado por cobertura: AFL++



La TUI de AFL++

american fuzzy lop 1.86b (test)		
process timing run time : 0 days, 0 hrs, 0 min, 2 sec last new path : none seen yet last uniq crash : 0 days, 0 hrs, 0 min, 2 sec last uniq hang : none seen yet		overall results cycles done : 0 total paths : 1 uniq crashes : 1 uniq hangs : 0
cycle progress now processing : 0 (0.00%) paths timed out : 0 (0.00%)	map coverage map density : 2 (0.00%) count coverage : 1.00 bits/tuple	
stage progress now trying : havoc stage execs : 1464/5000 (29.28%) total execs : 1697 exec speed : 626.5/sec	findings in depth favored paths : 1 (100.00%) new edges on : 1 (100.00%) total crashes : 39 (1 unique) total hangs : 0 (0 unique)	
fuzzing strategy yields bit flips : 0/16, 1/15, 0/13 byte flips : 0/2, 0/1, 0/0 arithmetics : 0/112, 0/25, 0/0 known ints : 0/10, 0/28, 0/0 dictionary : 0/0, 0/0, 0/0 havoc : 0/0, 0/0 trim : n/a, 0.00%		path geometry levels : 1 pending : 1 pend fav : 1 own finds : 0 imported : n/a variable : 0
[cpu: 92%]		

Demo

Fuzzing: arneses

- El objetivo es un binario
 - El fuzzer lo llama directamente pasando el input como fichero. Sencillo.
- ¿Y si en vez de un binario, queremos testear una librería?
 - No hay binario que llamar. Es necesario un construir un binario que simplemente llame a las funciones de la librería que queramos testear, como un wrapper
 - A ese binario se le llama *arnés* (*harness* en inglés)

Fuzzing: sanitizadores

- Un bug de memoria puede no matar el proceso.
 - El crash puede no ocurrir, saltar a un punto distinto del bug real, o darse el crash sin información de la causa.
 - Los sanitizadores lo detectan en tiempo de ejecución y hacen que el proceso muera con un mensaje detallado (dirección de memoria, trazas...).

**A****ASan***AddressSanitizer*

- Heap / stack buffer overflow
- Use-after-free
- Use-after-return
- Memory leaks

U**UBSan***UndefinedBehaviorSanitizer*

- Integer overflow
- Division by zero
- Null pointer dereference
- Shift out of bounds

M**MSan***MemorySanitizer*

- Lectura de memoria no inicializada
- Más costoso en rendimiento
- Cubre bugs que ASan/UBSan no ven

Fuzzing: Complementos a AFL++

afl-tmin

Minimiza un input que provoca un crash: de varios KB a, a veces, pocos bytes.

```
afl-tmin -i crash -o min -- ./bin @@
```

afl-cmin

Reduce el corpus al subconjunto mínimo que mantiene la misma cobertura total.

```
afl-cmin -i corpus -o min -- ./bin @@
```

afl-plot

Genera gráficas de cobertura, velocidad y crashes acumulados a lo largo de la sesión.

```
afl-plot out/session/ graficas/
```

Diccionarios

Tokens válidos de un formato (XML, JSON, HTML) para guiar mejor la mutación.

```
afl-fuzz -x dicts/xml.dict -i in -o out
```

¿Y luego, qué?



¿Y con la IA?

- La IA está democratizando **mucho** el uso de fuzzers, ya que está facilitando la generación de arneses y semillas.

[Home](#) > [Critical Information Infrastructures Security](#) > Conference paper

Sow Smarter, Not Harder: Evaluating LLM-Generated Seeds for Fuzzing Critical Infrastructure

Conference paper | First Online: 01 April 2026
pp 326–346 | [Cite this conference paper](#)

 [Save conference paper](#)

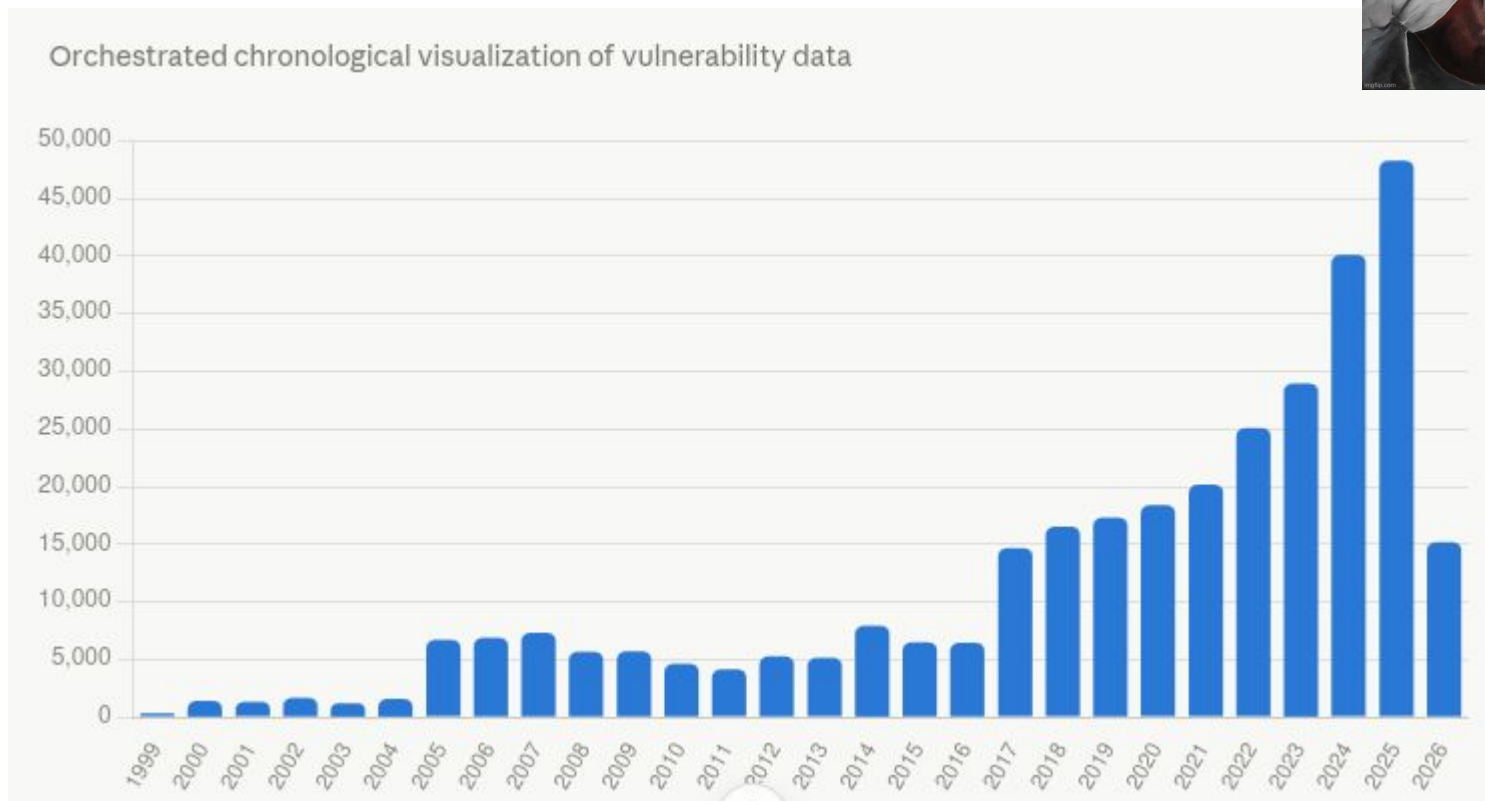
[Jorge Barredo](#) , [Maialen Eceiza](#), [Jose Luis Flores](#) & [Mikel Iturbe](#)



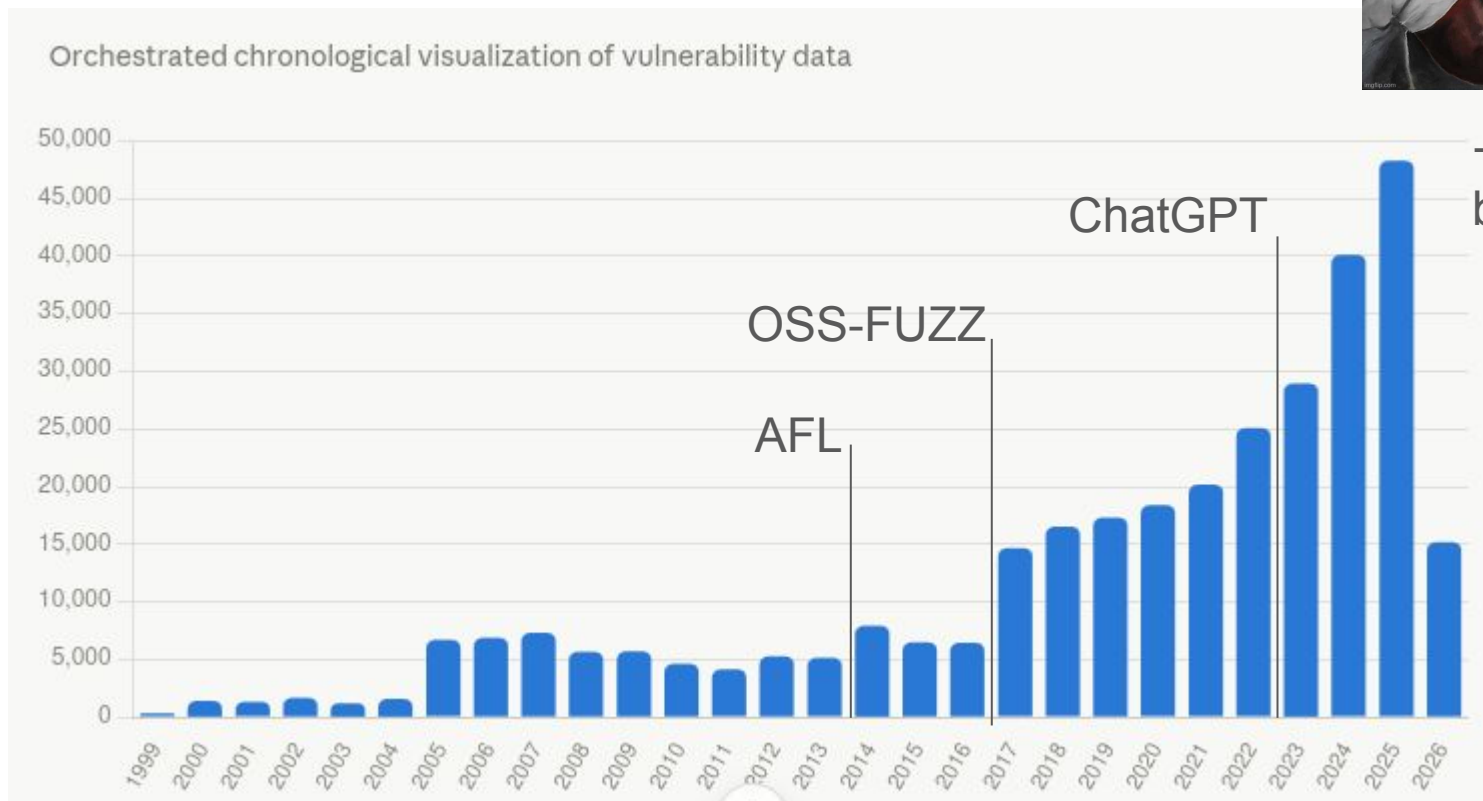
Critical Information Infrastructures Security
(CRITIS 2025)

[Access this chapter](#)

Identificación de vulnerabilidades: CVE



Identificación de vulnerabilidades: CVE



+ #CNAs,
bug bounty...

Aviso para la práctica

- Las sesiones de fuzzing son maratonianas (las sesiones duran días/semanas o incluso meses)
- La estrategia de mutación tiene un componente aleatorio
- Resultado: Es perfectamente posible (y lo más probable) que en la práctica no encontréis crashes.
 - Podéis seguir el contenido en casa y perseverar. Os aseguro que con tiempo, encontraréis los crashes ;-)
- Tenéis una línea principal de trabajo con fuzzing
 - Hay una serie de ejercicios pequeños que podéis realizar mientras el fuzzer echa humo.
- Hay bastante contenido. No os preocupéis.

Recursos adicionales

Recursos adicionales

- Gestión de vulnerabilidades
 - [NCSC Vulnerability management guide \(UK\)](#)
- Modelado de amenazas
 - [Cheatsheet Threat Modeling](#)
- Fuzzing
 - [The Fuzzing book](#)
 - [Fuzzing101](#)

Muchas gracias

Mikel Iturbe

miturbe@mondragon.edu

iturbe.info